



HANDBUCH · SCHRITT FÜR SCHRITT

Roblox- Erfolgsspiel

Deine Idee. Mein Fahrplan.
Von der leeren Seite bis zum Live-Betrieb.

19 Wochen bis zum Launch · getesteter Starter-Code · Checklisten & Vorlagen

Stand: 9. Oktober 2026

Plan, Zahlen und Quellen: siehe Kapitel „Lies mich zuerst“ und Anhang E

ÜBERSICHT

Inhaltsverzeichnis

Lies mich zuerst	5
So arbeitest du damit	5
Ehrliche Hinweise (bitte kurz lesen)	5
Meine Annahmen über dich (sag Bescheid, wenn etwas nicht stimmt)	6
Der Gesamtplan auf einen Blick	6
Symbole in diesem Handbuch	6
Kapitel 1 – Was auf Roblox wirklich funktioniert	7
1.1 Die Zahlen, die du kennen musst	7
1.2 Wie Spiele gefunden werden	7
1.3 Die 7 Gesetze eines erfolgreichen Spiels	8
1.4 Der Spielschleifen-Test	8
Kapitel 2 – Deine Idee prüfen und schärfen	10
2.1 Der 10-Fragen-Check	10
2.2 Marktscan in 3 Stunden	10
2.3 Dein Spielsteckbrief (ausfüllen!)	11
2.4 Die Rote-Linie-Prüfung (vor dem ersten Bauen)	12
Kapitel 3 – Einrichten: Konto, Studio, erstes Projekt	13
3.1 Konto absichern	13
3.2 Roblox Studio installieren	13
3.3 Erstes Projekt veröffentlichen	14
3.4 Game Settings: der wichtigste Schalter	15
3.5 Dateien und Sicherung	16
3.6 Studio-Gruppe anlegen (empfohlen)	16
3.7 Optional für Fortgeschrittene: Rojo + Git	16
3.8 Gate G-0: Fundament steht	16
Kapitel 4 – Das Starter-Kit einbauen	17
4.1 Wie das Kit funktioniert	17
4.2 Objekte anlegen	18
4.3 Der erste Test	19
4.4 Mehrspieler-Test (wichtig!)	20
4.5 Käufe einrichten (Game Pass und Developer Product)	20
4.6 Wie das Speichern funktioniert (und warum es sicher ist)	21
4.7 Was getestet wurde (und was nicht)	21
4.8 Gate G-1b: Kit läuft	21
Kapitel 5 – Dein Spiel um das Gerüst bauen	22

5.1 Den Platzhalter durch deinen Loop ersetzen	22
5.2 Die ersten 60 Sekunden (FTUE) bauen	23
5.3 Die Wirtschaft (Ökonomie) designen	24
5.4 Monetarisierung (Geld verdienen, ohne Spieler zu vergraulen)	25
5.5 Gründe, wiederzukommen (Retention-Mechaniken)	26
5.6 Soziales und Wachstum durch Freunde	26
5.7 Oberfläche und Handy	26
5.8 Performance (damit es auf schwachen Handys läuft)	27
5.9 Anti-Exploit (Schutz vor Schummeln)	27
5.10 Sicherheit für Kinder und Plattform-Regeln	27
5.11 Optik, Ton, KI-Werkzeuge	27
5.12 Sprachen	28

Kapitel 6 – Testen, Beta und Messen 29

6.1 Playtest: so läuft ein guter Test	29
6.2 Closed Beta aufsetzen (Woche 12)	29
6.3 Die Zahlen, die zählen (KPIs)	30
6.4 Der Verbesserungs-Kreislauf (jede Woche)	31
6.5 Diagnose: Was tun bei schlechten Zahlen?	32
6.6 Beta-Technik: der Härtestest	32
6.7 Beta-Umfrage (nach Tag 2 und Tag 7)	32
6.8 Die Gates G-2 und G-4 (Ampel-Entscheidung)	33

Kapitel 7 – Launch: Store-Seite, Marketing, Start 34

7.1 Die Store-Seite (Icon, Thumbnails, Titel)	34
7.2 Kurzvideos, die neugierig machen	35
7.3 Creator-Zusammenarbeit	35
7.4 Community (Discord ab 13)	36
7.5 Werbung (Roblox Ads): erst rechnen, dann kaufen	36
7.6 Der Launch-Plan (Runbook)	36
7.7 Soft Launch, dann Hard Launch	37
7.8 Letzte Prüfung (Recht und Plattform)	37

Kapitel 8 – Live-Betrieb und Wachstum 38

8.1 Der Wochenrhythmus (fester Kalender)	38
8.2 Event-Kalender 2027 (Vorschlag)	38
8.3 Update-Ideen nach Aufwand	38
8.4 Wachstums-Hebel (wenn die Zahlen stocken)	39
8.5 Community und Moderation	39
8.6 Team und Portfolio	39
8.7 Dich selbst schützen	39
8.8 Reviews nach 30, 60, 90 Tagen	39

Kapitel 9 – Geld, Steuern und Recht 40

9.1 Wie aus Spieler-Robux echtes Geld wird	40
9.2 Auszahlung (DevEx) Schritt für Schritt	40
9.3 Wie viel kommt von einem Euro an?	41
9.4 Szenarien: was ist realistisch?	41
9.5 Budget-Stufen (eigene Schätzungen)	42
9.6 Steuern und Recht in Deutschland: deine Fragenliste	42
9.7 Plattform-Regeln im Blick behalten	42
9.8 Prüfliste: Das konnte ich nicht bestätigen	42
Kapitel 10 – Dein 19-Wochen-Fahrplan	44
Wenn du in Verzug gerätst	45
Anhang A – Der Code des Starter-Kits	46
A.1 Config	46
A.2 Main	47
A.3 DataService	48
A.4 Analytics	52
A.5 EconomyService	53
A.6 DailyRewards	55
A.7 ShopService	56
A.8 BetaGate	58
A.9 Client	58
Anhang B – Checklisten zum Abhaken	62
B.1 Pre-Launch-Checkliste	62
B.2 Launch-Tag und die ersten 7 Tage	63
B.3 Wöchentliches KPI-Blatt (kopieren)	63
B.4 Monatliche Checks	63
Anhang C – Glossar	65
Anhang D – Hilfe holen	67
D.1 So meldest du ein Problem	67
D.2 Prompt-Bibliothek (kopieren, Klammern ausfüllen)	67
D.3 Typische Probleme	68
Anhang E – Quellen und Stand	69
Technische Hinweise zum Starter-Code	69

START

Lies mich zuerst

Du bringst die Idee mit. Ich liefere alles andere: den Fahrplan, die Einstellungen, den getesteten Grundcode, die Zahlen, die Vorlagen und die Entscheidungsregeln, damit aus deiner Idee ein Spiel wird, das auf Roblox wirklich funktionieren kann. Dieses Handbuch ist so gebaut, dass du es **von vorn nach hinten abarbeiten** kannst, ohne zu raten.

So arbeitest du damit

- Kapitel 1–2** (ca. 1 Woche): verstehen, was auf Roblox funktioniert, und deine Idee darauf prüfen und schärfen.
- Kapitel 3–4** (Woche 1–3): Konto, Studio und das Starter-Kit einrichten. Danach läuft schon ein kleines, speicherndes Spiel.
- Kapitel 5–6** (Woche 4–15): dein Spiel bauen, testen, verbessern. Jede Phase endet mit einem **Gate**: Weitermachen, Nachbessern oder Stoppen.
- Kapitel 7–9** (Woche 16–19 und danach): Launch, laufender Betrieb, Geld und Steuern.
- Kapitel 10 und Anhang**: Wochenplan zum Abhaken, Code, Checklisten, Glossar, Hilfe.

MACH JETZT — START

Fang heute mit **Kapitel 3, Schritt 3.1** an (Konto absichern). Das dauert 15 Minuten und blockiert nichts anderes. Den Rest der Woche 1 findest du in Kapitel 10.

Ehrliche Hinweise (bitte kurz lesen)

Thema	Was du wissen musst
Abbildungen	Ich kann in meiner Arbeitsumgebung weder Roblox Studio noch den Creator Hub öffnen. Deshalb sind alle Bilder von mir gezeichnete Skizzen mit nummerierten Markern, keine Original-Screenshots . Menünamen können bei dir leicht anders heißen, weil Roblox die Oberfläche oft ändert. Wenn dein Bildschirm abweicht: mach einen Screenshot und schick ihn mir , ich zeige dir die richtige Stelle.
Zahlen & Regeln	Die Roblox-Fakten stammen aus Suchergebnissen und Presseberichten (Stand 9. Okt 2026), nicht aus den offiziellen Creator-Hub-Seiten (die waren für mich gesperrt). Alles mit △ ist unbestätigt und steht in Kapitel 9.8 als Prüfliste. Prüfe diese Punkte in Woche 1.
Code	Der Starter-Code wurde in einer echten Luau-Umgebung mit nachgebauten Roblox-Diensten getestet (71 automatische Prüfungen, grün). Nicht getestet: die Oberfläche (Client), echte Roblox-Server, Handy-Leistung. Der erste Test in deinem Studio (Kapitel 4.3) ist deshalb Pflicht.
Erfolg	Niemand kann Erfolg garantieren: Der Median der Roblox-Entwickler verdient laut Presseberichten rund 1.500 \$ im Jahr. Dieses Handbuch maximiert deine Chancen und begrenzt dein Risiko, es verspricht kein Einkommen.
Steuern & Recht	Keine Steuer- oder Rechtsberatung. Ich nenne dir die Fragen, die du einem Steuerberater stellen musst (Kapitel 9.6).

Meine Annahmen über dich (sag Bescheid, wenn etwas nicht stimmt)

Thema	Annahme	Wenn anders ...
Zeit	15–20 Stunden pro Woche	Mehr Zeit: Phasen verkürzen. Weniger: Phasen strecken, Gates bleiben.
Erfahrung	Roblox Studio/Luau neu bis mittel	Profi: Kapitel 3 überfliegen.
Budget	1.500–3.000 € (Standard-Stufe, Kapitel 9.5)	0–500 €: alles selbst, keine Werbung. 8.000 €+: Freelancer + Ads.
Standort	Deutschland	Andere Länder: Kapitel 9.6 anpassen.
Zielgruppe	9–15 Jahre, weltweit, Englisch + Deutsch	Ältere Zielgruppe hat andere Regeln (Kapitel 5.10).

Der Gesamtplan auf einen Blick



Abb. 1 — 19 Wochen bis zum Launch (Start Mo 12.10.2026, Hard Launch Mo 15.02.2027). Jede Raute ist ein Gate: Go, Iterieren oder Stopp.

Puffer: Rechne mit +30 % Zeit (Weihnachten liegt mitten in Phase 3/4). Verschieben ist erlaubt, die **Reihenfolge der Gates nicht**.

Symbole in diesem Handbuch

Symbol	Bedeutung
MACH JETZT	Eine Aufgabe, die du jetzt erledigst.
TIPP	Abkürzung oder Profi-Trick.
ACHTUNG	Hier passieren die teuren Fehler.
PRÜFEN ⚠	Fakt ist nicht bestätigt. Vor dem Verlassen darauf selbst prüfen.
DEINE IDEE	Hier kommt DEINE Idee ins Spiel. Ich fülle das nicht für dich aus.
Gate (G-0 ... G-6)	Entscheidungspunkt: Weitermachen, Nachbessern oder Stoppen.

GRUNDLAGEN

Kapitel 1 – Was auf Roblox wirklich funktioniert

Bevor du eine Zeile baust, solltest du wissen, nach welchen Regeln der Markt spielt. Dieses Kapitel ist kurz, aber es entscheidet über alles Weitere.

1.1 Die Zahlen, die du kennen musst

Fakt	Zahl	Was das für dich heißt
Auszahlungen an Creator (12 Monate bis 30.06.2026)	ca. 1,7 Mrd. \$ (+ ca. 50 %)	Der Markt wächst und zahlt wirklich.
Top-10-Creator (Durchschnitt)	ca. 65,7 Mio. \$ je Creator	Ausreißer. Nicht als Planung verwenden.
Top-1.000 (Durchschnitt)	ca. 1,4 Mio. \$	Auch hier: Spitze der Pyramide.
Median unter > 42.000 DevEx-Teilnehmern	ca. 1.500 \$ pro Jahr	Das ist die ehrliche Basis. Dein Ziel: darüber landen, mit kleinem Risiko.
Hits wie Grow a Garden / Steal a Brainrot	bis ca. 25 Mio. gleichzeitige Spieler	Zeigt: simple Loops, in Monaten gebaut, können explodieren. Aber das sind Einzelfälle.

Quellen: Presseberichte zu Roblox' Entwicklerkonferenz 2026 (Dexerto, TweakTown, IBTimes) und PocketGamer.biz, siehe Anhang E. Die Originalzahlen von Roblox habe ich nicht direkt gesehen.

WISSEN — SO DENKT DER MARKT

Roblox ist wie ein riesiger Spielplatz mit Millionen Spielen. Die meisten Spiele werden nie gefunden. Wer gefunden wird, wird es, weil Spieler **wiederkommen**, Freunde mitbringen und Geld ausgeben. Das misst Roblox, und dafür baust du.

1.2 Wie Spiele gefunden werden

Roblox zeigt Spielern auf der Startseite «Empfohlen für dich». Laut Roblox Newsroom (Juni 2026) schaut dieser Empfehlungs-Algorithmus nicht mehr nur auf die letzten 7, sondern auf **28 Tage** und misst damit die **Langzeit-Retention**. Als Signale gelten u. a. Engagement, Wiederkehr, Freunde, die mitspielen, und Ausgaben.

- **Neue Spiele bekommen eine Chance**, aber nur, wenn Spieler in den ersten Tagen bleiben und wiederkommen.
- **Werbung allein hilft nicht**: Wer Besucher kauft, die nach 2 Minuten gehen, schadet seinem Signal.
- **Updates sind Marketing**: Ein Update ist ein Grund zurückzukommen und zeigt dem System, dass das Spiel lebt.



Abb. 2 — Retention-Kurve: Anteil der Spieler, die an Tag 1, 7, 28 nach dem ersten Besuch wiederkommen (Beispiel, keine Messung).

1.3 Die 7 Gesetze eines erfolgreichen Spiels

- 1 Ein Satz erklärt das ganze Spiel**
 «Ich sammle X, um Y zu verbessern und Z freizuschalten.» Verstehen Kinder das in 10 Sekunden nicht, versteht es der Algorithmus auch nicht, weil niemand bleibt. **Test:** Erkläre es einem 10-Jährigen.
- 2 Erste Belohnung in unter 60 Sekunden**
 Kurze Sessions sind das Problem: Spiele mit 0–6 Minuten Session haben laut GameAnalytics-Benchmark D1-Retention unter 10 % und kaum Umsatz. **Test:** Stoppuhr, Neuling-Konto, erste Belohnung?
- 3 Sessions von 10–20 Minuten**
 Roblox' Creator Rewards setzen u. a. 10 Minuten Spielzeit voraus $\triangle$. Entwirf Spannungsbögen von 10–20 Minuten mit Zwischenzielen.
- 4 Ein Grund für morgen**
 Tägliche Belohnung, Serien, Sammlungen, Timer, Events. Ohne Grund für morgen ist D28 nahe null.
- 5 Zusammen macht es mehr Spaß**
 Freunde einladen, Co-op-Bonus, Handeln, Rangliste, Angeben mit seltenen Dingen. Das ist Wachstum ohne Werbebudget.
- 6 Der Takt: jede Woche etwas Neues**
 Kleine Wochen-Updates, große Monats-Events. Das ist dein Marketing-Motor.
- 7 Läuft auf dem Handy**
 Ein Großteil der Spieler spielt mobil. Große Knöpfe, kurze Ladezeit, keine Abstürze auf schwachen Geräten.

ACHTUNG

Typische Anfängerfehler: (1) erst riesige Welt bauen, dann prüfen, ob es Spaß macht; (2) Shop vor dem Spielspaß; (3) keine Daten messen; (4) kurz vor Launch Werbung kaufen; (5) alles allein in einem Monat schaffen wollen.

1.4 Der Spielschleifen-Test

Jedes erfolgreiche Spiel funktioniert auf **vier Zeitskalen** gleichzeitig. Du prüfst deine Idee gleich in Kapitel 2 daran, und baust später jede Funktion so, dass sie mindestens eine dieser Ebenen stärkt.

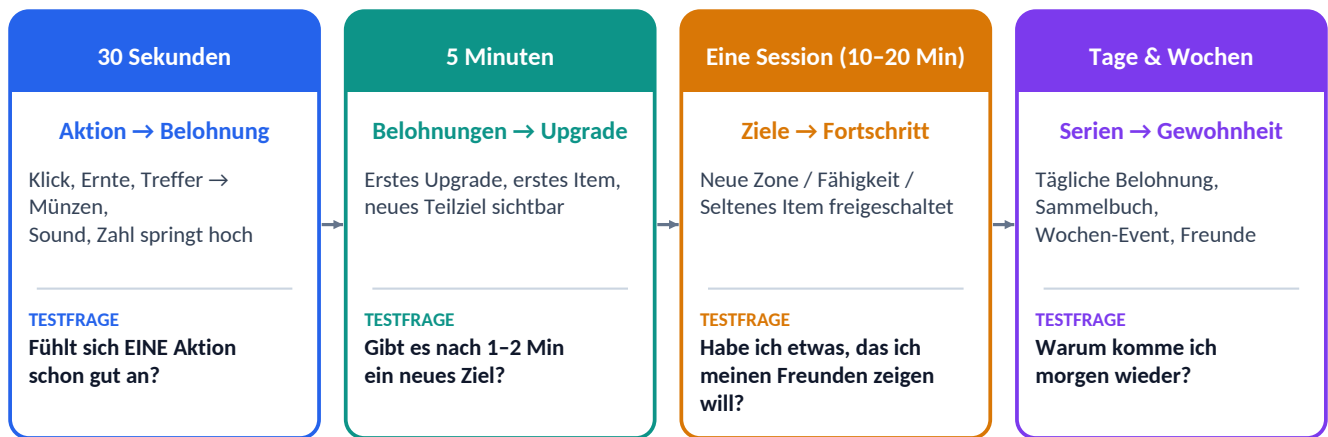


Abb. 3 — Die vier Zeitskalen einer Spielschleife und je eine Testfrage.

IDEE

Kapitel 2 – Deine Idee prüfen und schärfen

DEINE IDEE

Die Idee ist deine Sache. Dieses Kapitel hilft dir nicht beim Erfinden, sondern beim **Absichern**: Es zeigt dir, ob und wie die Idee auf Roblox tragen kann, bevor du Wochen investierst. Schreibe deine Antworten in den Spielsteckbrief (Vorlage unten).

2.1 Der 10-Fragen-Check

Bewerte jede Frage mit **2** (klar ja), **1** (teils) oder **0** (nein). Sei streng: Ein «teils» ist meistens ein «nein».

#	Frage	0 / 1 / 2
1	Kann ich den Loop in einem Satz erklären?	☐ 0 ☐ 1 ☐ 2
2	Kann ein Neuling in unter 60 Sekunden eine erste Belohnung bekommen?	☐ 0 ☐ 1 ☐ 2
3	Gibt es etwas zu sammeln oder zu verbessern , das nie ganz fertig wird?	☐ 0 ☐ 1 ☐ 2
4	Gibt es einen Grund, Freunde mitzubringen oder zu zeigen, was man hat?	☐ 0 ☐ 1 ☐ 2
5	Kann ich jede Woche Neues hinzufügen (Items, Zonen, Events), ohne alles umzubauen?	☐ 0 ☐ 1 ☐ 2
6	Kann ich eine spielbare Graybox-Version in 3 Wochen bauen?	☐ 0 ☐ 1 ☐ 2
7	Lässt sich das Spiel auf dem Handy mit einem Daumen bedienen?	☐ 0 ☐ 1 ☐ 2
8	Gibt es einen 10-Sekunden-Clip , der neugierig macht (für TikTok/Shorts)?	☐ 0 ☐ 1 ☐ 2
9	Gibt es 3 ähnliche, erfolgreiche Spiele (Beweis für Nachfrage) und ich kann eine Sache klar anders/besser ?	☐ 0 ☐ 1 ☐ 2
10	Ist die Idee frei von fremdem geistigen Eigentum (Marken, Figuren, Musik) und unbedenklich für Kinder ?	☐ 0 ☐ 1 ☐ 2

Summe	Bedeutung
17–20	Go. Weiter mit dem Steckbrief und dem Marktscan.
13–16	Nachschärfen. Löse die Fragen mit 0/1 gezielt (meist 2, 5 oder 9), dann neu bewerten.
unter 13	Idee ändern. Lieber jetzt eine Woche investieren als später 15.

2.2 Marktscan in 3 Stunden

Du suchst **nicht** nach einer Idee, sondern nach **Beweisen und Lücken** für deine.

- Öffne Roblox → **Entdecken (Discover)**. Suche mit 5–8 Begriffen, die zu deiner Idee passen. Notiere für **3–5 ähnliche Spiele** in der Tabelle unten.
- Schau dir **Besucherzahlen**, «**Gefällt mir**»-Quote und das **Datum des letzten Updates** an. Viele Besucher + altes Update = Nachfrage ohne Pflege = Chance.
- Spiele jedes Spiel **15 Minuten als Neuling** (neues Konto). Stoppe: Wann kam die erste Belohnung? Wo hast du aufgehört?
- Lies die **Kommentare/Bewertungen**: Was nervt die Spieler? Das ist deine Lücke.

5. Schreibe auf: **Was machst du anders in einem Satz?** Wenn dir nichts einfällt: zurück zu Frage 9.

Spiel	Besucher / Gleichzeitig	Letztes Update	Erste Belohnung nach	Was nervt?	Meine Lücke

TIPP

Spiele mit **vielen Spielern und sehr alten Updates** sind die besten Zeichen: Die Nachfrage ist da, die Pflege fehlt. Genau dort schlägst du mit einem guten Wochentakt.

2.3 Dein Spielsteckbrief (ausfüllen!)

Speichere ihn als Datei docs/concept/one-pager.md in diesem Repo oder schreibe ihn auf Papier. Ohne Steckbrief startest du Kapitel 3 nicht zu Ende.

Feld	Deine Antwort
Arbeitstitel	
Ein-Satz-Loop (Gesetz 1)	
Zielgruppe (Alter, Interessen)	
30 Sekunden: Was tut der Spieler, was bekommt er?	
5 Minuten: Welches Upgrade/Item kommt als Nächstes?	
Session (10–20 Min): Was ist das «Wow»?	
Tage/Wochen: Warum kommt er morgen wieder?	
Erste Belohnung nach ... Sekunden (Ziel: unter 60)	
Sozialer Haken (Freunde, Handel, Rangliste, Angeben)	
Was wird nie fertig? (Sammeln/Prestige)	
3 Konkurrenzspiele und mein Unterschied	
Art-Stil (z. B. Low-Poly, bunt)	
3 Monetarisierungs-Ideen (nur Komfort/Tempo/Optik)	
Größtes Risiko und wie ich es früh teste	

2.4 Die Rote-Linie-Prüfung (vor dem ersten Bauen)

- ☐ **Kein fremdes IP:** Keine Marken, Charaktere, bekannte Memes mit Rechteinhaber, keine Musik/Bilder von außerhalb. Inspiration für *Mechaniken* ist okay, Namen/Assets/Texte müssen eigenständig sein.
- ☐ **Kinderschutz:** Keine Räume/Orte mit Bar/Club/Schlafzimmer-Charakter (Roblox schränkt solche Inhalte auf 17+ ein), keine Gewalt-Darstellung, die über die Alterseinstufung hinausgeht.
- ☐ **Keine Glücksspiel-Mechanik mit Echtgeld-Charakter** (z. B. Lootboxen ohne Quotenangabe). Details in Kapitel 5.4.
- ☐ **Keine Daten von Kindern sammeln** und keine Aufforderungen, Roblox zu verlassen (Chat, Social Media, Discord) im Spiel.
- ☐ **Ehrliche Darstellung:** Thumbnails/Beschreibung zeigen nur, was es im Spiel wirklich gibt.

MACH JETZT — GATE G-1 – bis spätestens Woche 3

Fülle den Check (2.1), den Marktskan (2.2) und den Steckbrief (2.3) aus. Das ist **Gate G-1**: Score mindestens 17, Loop in einem Satz erklärbar und mindestens 3 von 5 Testern aus der Zielgruppe wollen es spielen («Papier-Test»: Erkläre die Idee in 2 Minuten, frage «Würdest du das spielen?»). Bei Kindern nur mit Einverständnis der Eltern.

SETUP · WOCHE 1

Kapitel 3 – Einrichten: Konto, Studio, erstes Projekt

Hier legst du das Fundament. Alles ist kostenlos. Nimm dir einen Abend dafür und hake jeden Schritt ab. **Wichtig:** Roblox Studio gibt es offiziell nur für Windows und macOS (nicht für Linux/Chromebook).

3.1 Konto absichern

3.1 Roblox-Konto und Sicherheit

1. Öffne **roblox.com** und melde dich an (oder registriere dich). Nutze ein Konto, das **nur dir** gehört und dessen E-Mail du kontrollierst.
2. **Einstellungen** (Zahnrad) → **Sicherheit: 2-Schritt-Verifizierung** einschalten (Authenticator-App ist am sichersten). Danach eine Konto-PIN setzen.
3. **E-Mail bestätigen** (Einstellungen → Konto-Infos).
4. **Alters-/ID-Verifizierung** durchführen, falls angeboten. Für die Auszahlung (DevEx) ist sie voraussichtlich nötig ⚠ (Prüfliste F3).

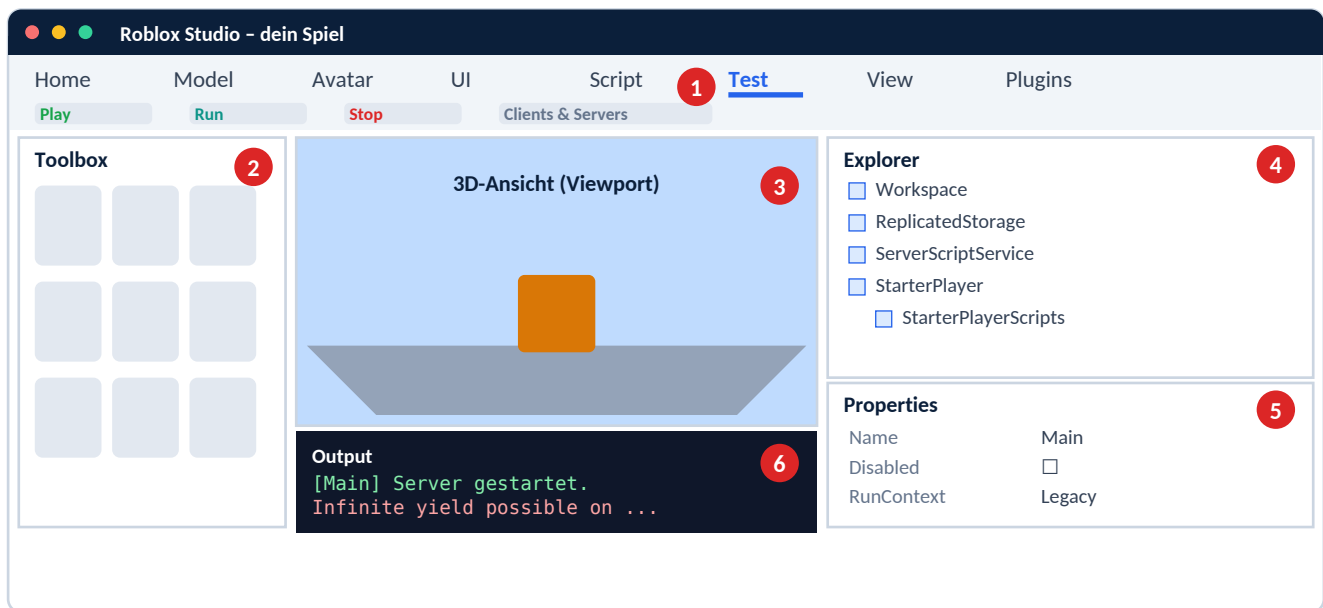
ACHTUNG

Gib Passwort und Verifizierungs-Codes **niemals** weiter, auch nicht an Helfer, Freelancer oder «Admins». Wer Zugriff braucht, bekommt eine **Rolle in deiner Gruppe** (3.6), nie dein Passwort.

3.2 Roblox Studio installieren

3.2 Studio herunterladen und starten

1. Öffne **create.roblox.com** (der «Creator Hub») und melde dich an. Klicke auf **«Start Creating»** bzw. «Roblox Studio herunterladen».
2. Installer ausführen, danach Studio starten und mit deinem Konto anmelden.
3. Wähle auf der Startseite **«New»** (Neu) → Vorlage **«Baseplate»** (leere Bodenplatte).
4. Blende die wichtigen Fenster ein: Tab **View** → **Explorer, Properties, Output** (und **Toolbox**). Ohne Explorer und Output arbeitest du blind.



- 1 Registerkarten.** «Test»: Play (F5) startet dein Spiel, «Stop» beendet es. «View» blendet Fenster ein.
- 2 Toolbox.** Fertige Modelle aus dem Creator Store (Lizenz & Qualität prüfen!).
- 3 Viewport.** Hier baust du die Welt: Mausrad = Zoom, Rechtsklick halten + WASD = fliegen.
- 4 Explorer.** Baum aller Objekte. Hier legst du Scripts an (Maus drauf → «+»).
- 5 Properties.** Einstellungen des markierten Objekts.
- 6 Output.** Meldungen deines Codes. Rot = Fehler → genau diese Zeile schickst du mir.

Abb. 4 — Roblox Studio (Skizze, kein Original-Screenshot): die sechs Bereiche, die du ab jetzt ständig brauchst.

TIPP

Merke dir 4 Tasten: F5 = Spiel starten (Play) · Shift+F5 = beenden · F8 = Run (Server ohne Spielfigur) · Strg+S = speichern. Und: Immer das **Output-Fenster** offen lassen. Rote Zeilen sind Fehlermeldungen. Genau die brauchst du, wenn du mir schreibst.

3.3 Erstes Projekt veröffentlichen

3.3 Das Spiel einmal «publishen» (privat)

1. Menü **File** → **Publish to Roblox As...** (Datei → Als Spiel veröffentlichen).
2. **Name:** Arbeitstitel. **Ersteller:** deine **Gruppe** (falls schon angelegt, sonst später umziehen). **Genre** und **Geräte** (Computer, Handy, Tablet) wählen. **Sichtbarkeit: Privat.**
3. Klicke auf **Create**. Danach siehst du das Spiel im Creator Hub unter **Creations**.

Abb. 5 — Der Veröffentlichen-Dialog (Skizze). Anfangs immer privat.

ACHTUNG

Handy immer aktivieren. Wenn du «Handy» abwählst, kann ein Großteil der Spieler dein Spiel nicht öffnen.

3.4 Game Settings: der wichtigste Schalter

3.4 DataStore-Zugriff im Studio erlauben

1. Tab **Home** → **Game Settings**.
2. Links **Security** öffnen. Schalter «**Enable Studio Access to API Services**» auf **AN**. Das erlaubt Speichern/Laden schon im Studio-Test.
3. «**Allow HTTP Requests**» bleibt **AUS**.
4. Unter **Basic Info** kannst du Name, Beschreibung, Genre und Geräte ändern. Unter **Permissions** siehst du die Sichtbarkeit (Privat/Öffentlich).
5. **Save** klicken.

1. Menüpunkt **Security** in der linken Spalte öffnen.
2. Schalter **AN**: dann funktionieren Speichern/Laden schon im Studio-Test.
3. HTTP-Requests **AUS** lassen.

Abb. 6 — Game Settings → Security (Skizze). Ohne diesen Schalter speichert dein Studio-Test nichts.

3.5 Dateien und Sicherung

- Speichere regelmäßig mit **Strg+S**. Studio legt zusätzlich Autosaves an.
- Im Creator Hub gibt es einen **Versionsverlauf** des veröffentlichten Spiels. Damit kannst du zu einer älteren Version zurück (genauen Menünamen im Creator Hub nachsehen).
- Der Code dieses Handbuchs liegt zusätzlich als Dateien im Repo `game/src/`. Das ist deine Sicherheitskopie.

3.6 Studio-Gruppe anlegen (empfohlen)

3.6 Eine Gruppe für dein Studio

1. Auf `roblox.com` → **Communities/Gruppen** → **Create**. Name = dein Studio-Name. (Die Gründung kann Robux kosten, Preis steht im Dialog.)
2. Veröffentliche deine Spiele **unter der Gruppe**. Vorteile: Teamrollen, später Auszahlungs-Aufteilung, das Spiel gehört nicht einer einzelnen Person.
3. Nutze die Gruppe später auch für die **Closed Beta** (Kapitel 6.2).

PRÜFEN

Auszahlung aus Gruppen: Wie Gruppen-Einnahmen ausgezahlt werden und wer dafür berechtigt sein muss, solltest du im Creator Hub nachlesen, bevor du die Gruppe zum Eigentümer machst [△](#) (Prüfliste F3).

3.7 Optional für Fortgeschrittene: Rojo + Git

Wenn du später mit mir (Claude Code) im Repo arbeitest, kann Code als **Dateien** liegen und per **Rojo** ins Studio synchronisiert werden. Das Repo enthält dafür schon `game/default.project.json`. Für den Anfang ist es **nicht nötig**: Du kannst den Code per Copy&Paste ins Studio einfügen (Kapitel 4).

```
rokit init                                # Toolchain-Manager (oder: aftman)
rokit add rojo-rbx/rojo                  # Rojo
rojo serve game/default.project.json     # dann im Studio das Rojo-Plugin → Connect
# Versionsnummern und Befehle gegen die aktuelle Rojo-Doku prüfen
```

3.8 Gate G-0: Fundament steht

- ☐ Konto abgesichert (2FA, E-Mail, ggf. ID-Verifizierung)
- ☐ Studio installiert, Output + Explorer sichtbar
- ☐ Baseplate-Projekt veröffentlicht (privat, Handy aktiv)
- ☐ «Enable Studio Access to API Services» = AN
- ☐ Gruppe angelegt (oder bewusst später)
- ☐ Steuerberater-Termin gebucht (Kapitel 9.6)
- ☐ Prüfliste F1–F7 (Kapitel 9.8) im Creator Hub nachgesehen

TECHNIK · WOCHE 3

Kapitel 4 – Das Starter-Kit einbauen

Jedes erfolgreiche Spiel braucht dieselbe unsichtbare Technik: **sicheres Speichern, ein Server, der entscheidet, Käufe, die nie doppelt zählen, Anti-Spam, Messung.** Das ist langweilig, aber hier gehen Spiele kaputt. Das Starter-Kit liefert dir all das fertig. Dein Spiel kommt **obendrauf**.

4.1 Wie das Kit funktioniert

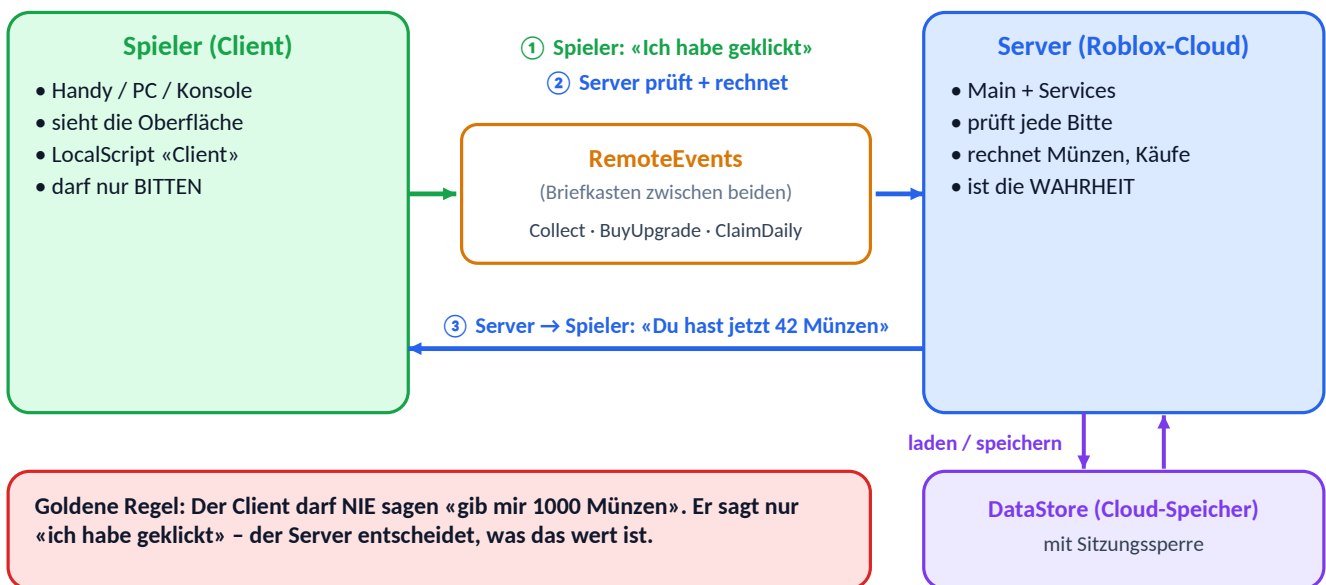


Abb. 7 — Die Grundregel von Roblox-Spielen: Der Server entscheidet, der Client darf nur bitten.

Als Platzhalter-Spiel enthält das Kit einen minimalen **Klick-Sammel-Loop**: SAMMELN gibt Münzen, UPGRADE macht jeden Klick wertvoller, TÄGLICH gibt eine tägliche Belohnung, «2x MÜNZEN» ist ein Game Pass. **Das ist nur Gerüst.** Wie du es durch deinen eigenen Loop ersetzt, steht in Kapitel 5.1.

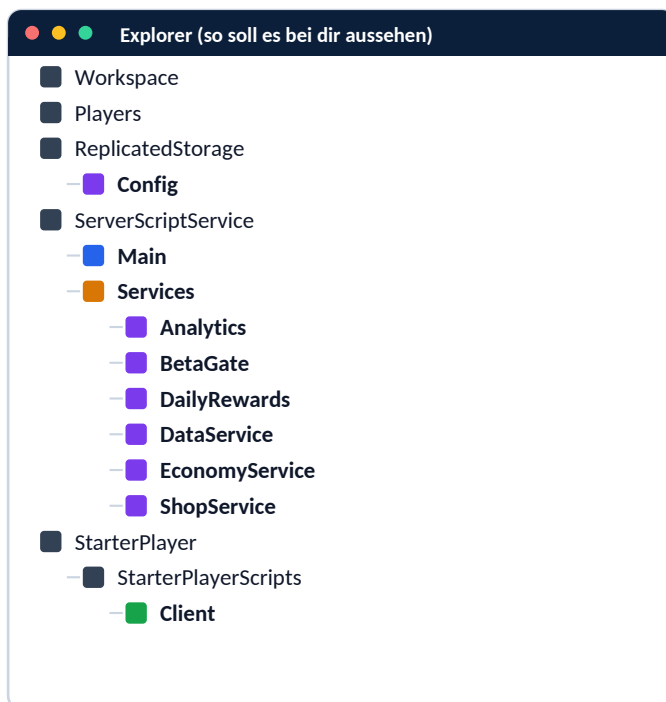


Abb. 8 — So sieht das Starter-Kit im Spiel aus (Skizze, Querformat-Handy).

4.2 Objekte anlegen

Du legst insgesamt **10 Objekte** an und fügst Code ein. Gehe die Tabelle von oben nach unten durch. Die Reihenfolge ist egal, nur die **Namen müssen exakt** stimmen.

Name	Typ	Ort im Explorer	Code in
Config	ModuleScript	ReplicatedStorage	Anhang A.1
Main	Script	ServerScriptService	Anhang A.2
Services	Folder	ServerScriptService	(leer, nur Ordner)
DataService	ModuleScript	ServerScriptService → Services	Anhang A.3
Analytics	ModuleScript	... → Services	Anhang A.4
EconomyService	ModuleScript	... → Services	Anhang A.5
DailyRewards	ModuleScript	... → Services	Anhang A.6
ShopService	ModuleScript	... → Services	Anhang A.7
BetaGate	ModuleScript	... → Services	Anhang A.8
Client	LocalScript	StarterPlayer → StarterPlayerScripts	Anhang A.9



Farben = Objekt-Typ

- Script – läuft auf dem SERVER
- ModuleScript – Werkzeugkasten (wird mit require geladen)
- LocalScript – läuft auf dem Handy/PC des Spielers
- Folder – nur Ordner zum Sortieren
- Dienste (Services) – bringt Roblox mit

Merke dir 3 Regeln

- Geheimes & Wichtiges (Daten, Käufe) nur in ServerScriptService – der Spieler kann diesen Ordner NIE lesen.
- Config liegt in ReplicatedStorage, weil Server UND Client sie lesen.
- Den Ordner «Remotes» legt Main beim Start selbst an.

Namen müssen EXAKT stimmen (Groß/Klein):
«DataService», nicht «dataservice».

Abb. 9 – So soll dein Explorer aussehen, wenn alles angelegt ist (Skizze).

4.2 Ein Objekt anlegen: so geht's

1. Im **Explorer** mit der Maus über den Dienst fahren (z. B. **ServerScriptService**) → rechts erscheint ein kleines «+».
2. Typ wählen (**Script**, **ModuleScript**, **LocalScript** oder **Folder**).
3. Neues Objekt anklicken, **F2** drücken, Namen eintippen, Enter.
4. **Doppelklick** öffnet den Editor. Alles markieren (**Strg+A**), alten Beispielcode löschen, neuen Code einfügen (**Strg+V**).
5. **Strg+S**.

TIPP

Code kopieren: Du findest jeden Code-Block in Anhang A dieses PDFs **und** als fertige Dateien im Repo unter `game/src/`. Am zuverlässigsten ist die Datei (PDF-Kopien können Zeilenumbrüche verändern; Luau stört das normalerweise nicht). Du kannst mich auch bitten, dir die Dateien einzeln im Chat auszugeben.

4.3 Der erste Test

4.3 Spiel starten und prüfen

1. Tab **Test** → **Play** (oder **F5**). Du siehst die Oberfläche des Kits.
2. Im **Output** muss stehen: `[Main] Server gestartet`.
3. Tippe auf **SAMMELN**: Münzen steigen. Kaufe ein **UPGRADE** (kostet 10): jeder Klick bringt jetzt mehr. Tippe auf **TÄGLICH**: +50 Münzen, danach grau («MORGEN»).
4. Beende mit **Stop**. Starte erneut (**F5**): **Münzen und Level sind noch da** (wenn Kapitel 3.4 erledigt ist).
5. Ohne API-Zugriff steht im Output eine Warnung **Testmodus OHNE Speichern**. Das ist gewollt und zeigt dir, dass der Schalter aus ist.

Du siehst ...	Das bedeutet	Lösung
«Infinite yield possible on ReplicatedStorage:WaitForChild('Config')»	Objekt «Config» fehlt oder heißt anders.	Name/Ort prüfen (Abb. Explorer), Groß/Klein beachten.
«Services is not a valid member of ...»	Ordner «Services» fehlt unter <code>ServerScriptService</code> .	Ordner anlegen, Module hineinziehen.
Buttons erscheinen, Klick tut nichts	Server-Script «Main» fehlt/Fehler oder Remotes fehlen.	Output nach roten Zeilen durchsuchen; Main muss ein Script (kein <code>LocalScript</code>) sein.
Keine Oberfläche	«Client» ist falsch platziert oder kein <code>LocalScript</code> .	Muss unter <code>StarterPlayer</code> → <code>StarterPlayerScripts</code> liegen.
«Testmodus OHNE Speichern»	API-Zugriff im Studio aus.	Kapitel 3.4, danach Studio neu starten.
Rote Zeile mit «attempt to index nil»	Ein Objekt wird nicht gefunden.	Zeile mit Dateiname + Zahl kopieren und mir schicken.

4.4 Mehrspieler-Test (wichtig!)

4.4 Mit mehreren Spielern testen

1. Tab **Test** → **Clients and Servers: Local Server**, Spieleranzahl **2**, **Start**. Es öffnen sich ein Server- und zwei Spielerfenster.
2. In beiden Fenstern sammeln, upgraden. Prüfe: Jeder sieht nur **seine eigenen** Münzen; die Rangliste oben rechts zeigt beide.
3. Ein Spielerfenster schließen und neu starten: Stand bleibt erhalten.

4.5 Käufe einrichten (Game Pass und Developer Product)

Der Unterschied: Ein **Game Pass** kauft man **einmal für immer** (z. B. «2x Münzen»). Ein **Developer Product** kann man **beliebig oft** kaufen (z. B. «500 Münzen»).

- 1 Icon hochladen (PNG/JPG, quadratisch).
- 2 Name + kurze Beschreibung, die den NUTZEN nennt.
- 3 Startpreis – später per Test anpassen (Kapitel 5.4).
- 4 **Zum Verkauf** einschalten. Danach die ID kopieren (steht in der Adresszeile / auf der Pass-Seite) und in `Config.GAME_PASSES.DoubleCoins` eintragen.

Abb. 10 — Einen Game Pass im Creator Hub anlegen (Skizze, Menünamen können abweichen).

1. Creator Hub → dein Spiel → **Monetarisierung** (Monetization) → **Passes** → **Create a Pass**: Icon, Name, Beschreibung, Preis; **Zum Verkauf anbieten** einschalten.
2. Die ID (Zahl in der Adresszeile der Pass-Seite) in `Config.GAME_PASSES.DoubleCoins` eintragen. Gleiches für **Developer Products** in `Config.PRODUCTS.Coins500.id` (dort stellst du auch ein, wie viele Münzen es gibt).
3. **Spiel veröffentlichen** (File → Publish to Roblox) und im veröffentlichten Spiel prüfen: «2x MÜNZEN» öffnet das Kaufmenü.
4. Zum Test **zuerst ein Produkt mit 1–5 Robux** anlegen und die Abbuchung/Gutschrift kontrollieren. Ob Käufe im Studio-Test echte Robux kosten, steht in der Creator-Hub-Doku [△](#) nach, bevor du in Studio kaufst.

ACHTUNG

Nie eine Belohnung auf dem Client auslösen («Kauf fertig → gib Münzen»). Das Kit vergibt Belohnungen nur im **Server-Callback ProcessReceipt** und merkt sich jede Kauf-ID, damit ein Kauf **nicht doppelt** zählt.

4.6 Wie das Speichern funktioniert (und warum es sicher ist)

Das Kit speichert beim Verlassen, alle 60 Sekunden, bei Käufen und beim Server-Shutdown. Jeder Spieler hat eine **Sitzungssperre**: Nur ein Server darf gleichzeitig schreiben. Das verhindert verlorene Spielstände, wenn jemand schnell den Server wechselt.

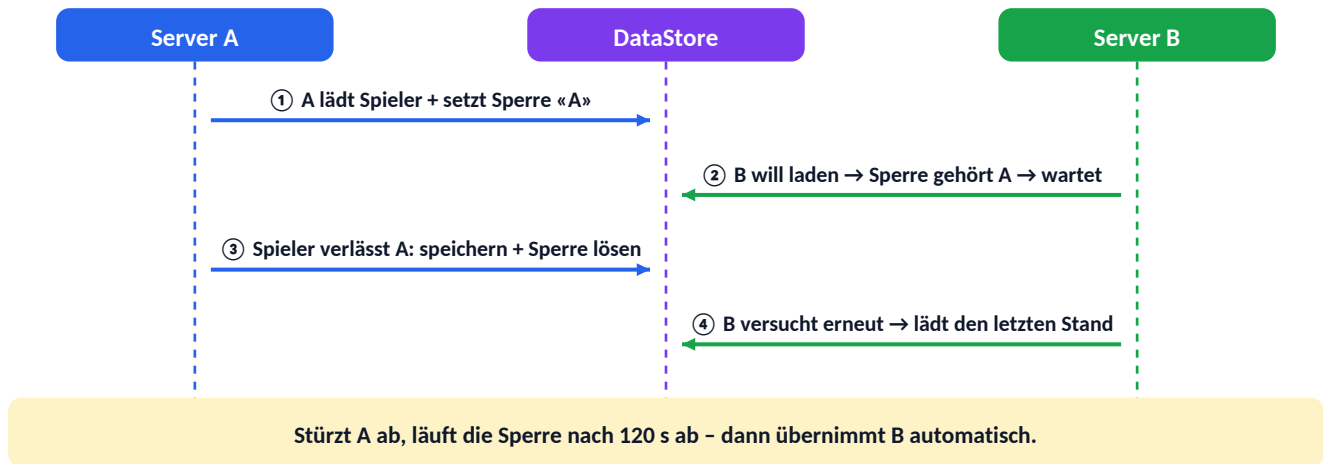


Abb. 11 — Sitzungssperre: warum Spielerdaten beim Server-Wechsel nicht verloren gehen.

4.7 Was getestet wurde (und was nicht)

Die Server-Logik läuft in einer echten Luau-Umgebung (nicht in Roblox selbst) mit nachgebauten Roblox-Diensten. Zusätzlich habe ich **absichtlich 5 Fehler in den Code eingebaut** (z. B. Rate-Limit entfernt, Doppelkauf-Schutz entfernt) und geprüft, dass die Tests sie finden. Das tun sie.

Getestet (71 Prüfungen, alle grün)	Nicht getestet (prüfst du selbst)
• Beitritt, Laden, Speichern, Wiederbeitritt • Auto-Save, Server-Shutdown • Server-Wechsel und Sitzungssperre, abgelaufene Sperre • DataStore-Fehler und Wiederholungen, Studio-Testmodus • Münzen, Upgrades, Rate-Limit, ungültige Werte • Käufe: keine Doppelbelohnung, Fehler beim Speichern • Game Pass (Multiplikator), tägliche Belohnung inkl. Gnadentag • Closed-Beta-Zugang	• Oberfläche (Client) und Layout auf echten Handys • Echte Roblox-Server, Latenz und DataStore-Limits • Kaufdialoge mit echten Robux • Analytics-Aufrufe (Namen in der Doku prüfen; sie sind absichtlich in <code>pcall</code>, damit nichts abstürzt) • Spielgefühl, Balance, Performance auf schwachen Geräten

Die Tests liegen im Repo unter `game/tests/run.luau` (Anleitung: `game/tests/README.md`).

4.8 Gate G-1b: Kit läuft

- ☐ Alle 10 Objekte angelegt, Namen stimmen
- ☐ F5-Test: «[Main] Server gestartet.», Oberfläche sichtbar
- ☐ Münzen/Level bleiben nach Stop → Play erhalten
- ☐ Mehrspieler-Test mit 2 Spielern erfolgreich
- ☐ Game Pass und 1 Testprodukt angelegt, IDs in Config
- ☐ Im veröffentlichten Spiel öffnet «2x MÜNZEN» den Kaufdialog

BAUEN · WOCHE 4–11

Kapitel 5 – Dein Spiel um das Gerüst bauen

Jetzt wird es dein Spiel. Dieses Kapitel ist dein Bau-Handbuch: wie du den Platzhalter ersetzt, was du in welcher Reihenfolge baust und worauf du bei jedem Teil achtest. Prinzip: **erst Spaß (Woche 4–6), dann Tiefe (7–11), dann Politur**. Nie umgekehrt.

5.1 Den Platzhalter durch deinen Loop ersetzen

Teil im Kit	Was er jetzt tut	So wird er zu deinem Spiel
SAMMELN (Remote «Collect»)	Klick = Münzen	Ersetze durch DEINE Kernaktion (ernten, kämpfen, bauen, fangen ...). Bleibt serverseitig und rate-limitiert.
UPGRADE (Remote «BuyUpgrade»)	Mehr Münzen pro Klick	Dein Fortschrittssystem: Fähigkeiten, Werkzeuge, Haustiere, Zonen. Preise kommen aus Config .
Münzen (coins)	Einziges Währung	Entscheide: eine Währung (einfacher) oder zwei (Soft + Premium)? Start: eine .
TÄGLICH	Serie + Belohnung	Behalten. Passe Config.DAILY_REWARDS an; später mit Items statt nur Münzen.
Spielerdaten (DEFAULT_DATA)	Münzen, Level, Serie	Neues Feld = hier eintragen. Versionsnummer erhöhen, wenn du Felder umbaut.
Client-UI	4 Knöpfe per Code	Später echtes UI-Design (Studio → UI-Editor). Die Server-Logik bleibt gleich .

Rezept: Eine neue Funktion in 7 Schritten (immer gleich)

1. **Zahlen in Config** auslagern (Preise, Abklingzeiten, Belohnungen).
2. **Daten-Feld** in **DEFAULT_DATA**.
3. **Server-Funktion** mit Prüfung der Eingaben und Abklingzeit.
4. **RemoteEvent** in Main anlegen.
5. **UI-Knopf** im Client, der den Remote feuert.
6. **Analytics-Ereignis** (damit du es messen kannst).
7. **Test**: normaler Ablauf + Missbrauch (Remote mit Unsinn feuern).

Beispiel «Pflanze gießen» (Schritte 1–5 im Überblick). Wichtig ist das Muster: prüfen, Abklingzeit, ändern, belohnen.

```
-- 1) Config.luau: Zahlen auslagern
Config.PLOT_COUNT = 6
Config.WATER_COOLDOWN = 3 -- Sekunden
Config.WATER_REWARD = 5
-- und in DEFAULT_DATA ein neues Feld: waterCount = 0

-- 2) Neue Datei Services/PlantService.luau (ModuleScript)
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local Config = require(ReplicatedStorage.WaitForChild("Config"))
local DataService = require(script.Parent.DataService)
local EconomyService = require(script.Parent.EconomyService)

local PlantService = {}
local lastAction = {} -- [Player] = Zeitpunkt der letzten Aktion

function PlantService.start(remotes)
    remotes.Water.OnServerEvent:Connect(function(player, plotId)
        -- a) Eingaben prüfen: nie dem Client vertrauen
        if type(plotId) ~= "number" or plotId % 1 ~= 0 or plotId < 1 or plotId > Config.PLOT_COUNT then
            return
        end
        local data = DataService.get(player)
        if not data then
            return
        end
        -- b) Abklingzeit auf dem Server erzwingen
        local now = os.clock()
        if now - (lastAction[player] or -1000) < Config.WATER_COOLDOWN then
            return
        end
        lastAction[player] = now
        -- c) Zustand ändern + belohnen (alles serverseitig)
        data.waterCount += 1
        EconomyService.addCoins(player, Config.WATER_REWARD, "water")
    end)
end

return PlantService

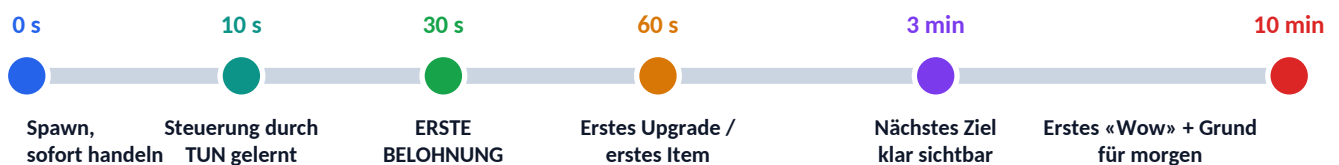
-- 3) In Main.server.luau: "Water" zur Remote-Liste hinzufügen und PlantService.start(remotes) aufrufen
-- 4) Client: button.Activated:Connect(function() remotes.Water:FireServer(1) end)
```

DEINE IDEE

Hier ist deine Arbeit: Beschreibe mir deinen Loop in 5 Sätzen (Aktion, Belohnung, erstes Upgrade, erstes Ziel, soziales Element). Ich schreibe dir daraus die Server- und Client-Teile nach diesem Rezept, inklusive Config-Werten und Testplan.

5.2 Die ersten 60 Sekunden (FTUE) bauen

FTUE = First-Time User Experience, also das, was ein Neuling in den ersten Minuten erlebt. Hier gewinnst oder verlierst du die Mehrheit deiner Spieler.



Dos & Don'ts der ersten Minuten

- | | |
|---|--|
| ✓ Pfeil/Highlight auf das erste Ziel | ✗ Textwände, Tutorial-Dialoge zum Wegklicken |
| ✓ Belohnung mit Sound + Zahl + Bewegung | ✗ Shop-Pop-ups in den ersten 3 Minuten |
| ✓ Ein Ziel nach dem anderen zeigen | ✗ Ladebildschirm länger als nötig |

Abb. 12 — Zeitplan der ersten Minuten (Ziele). Mit der Stoppuhr am Neuling-Konto nachmessen.

- ☐ Der Spieler darf in den ersten 3 Sekunden **handeln** (kein Warten auf Ladebalken/Dialog).
- ☐ Die **erste Belohnung** kommt in unter 30 Sekunden, mit **Sound, Zahl, Bewegung**.
- ☐ Es gibt **immer genau ein klares nächstes Ziel** (Pfeil, Leuchten, Text mit max. 5 Wörtern).
- ☐ Das **erste Upgrade** ist nach 30–60 Sekunden erreichbar.
- ☐ **Keine** Shop-Fenster in den ersten 3 Minuten.
- ☐ Test: Gib das Spiel jemandem ohne Erklärung. Misst du **keine Frage** in den ersten 2 Minuten, ist es gut.

5.3 Die Wirtschaft (Ökonomie) designen

Die Wirtschaft entscheidet, ob sich Fortschritt gut anfühlt. Zwei Fragen: **Wie schnell verdient man?** (Quellen) und **Wofür gibt man aus?** (Senken).

Quellen (Münzen kommen rein)	Senken (Münzen gehen raus)
• Kernaktion (Klick/Ernte/Kampf) • Tägliche Belohnung, Quests, Events • Käufe mit Robux (Developer Products) • Freunde-Bonus, Multiplikatoren	• Upgrades (steigende Preise) • Neue Zonen/Fähigkeiten freischalten • Rebirth/Prestige (Reset gegen Dauer-Bonus) • Kosmetik, Sammel-Items, Haustier-Slots

Das Kit nutzt eine einfache, bewährte Formel für Preise: **Preis = Basispreis × Wachstum^{Level}** (Standard: $10 \times 1,15^{\text{Level}}$). Der Ertrag wächst linear (+1 pro Level). So wird Fortschritt zuerst schnell und später langsamer.

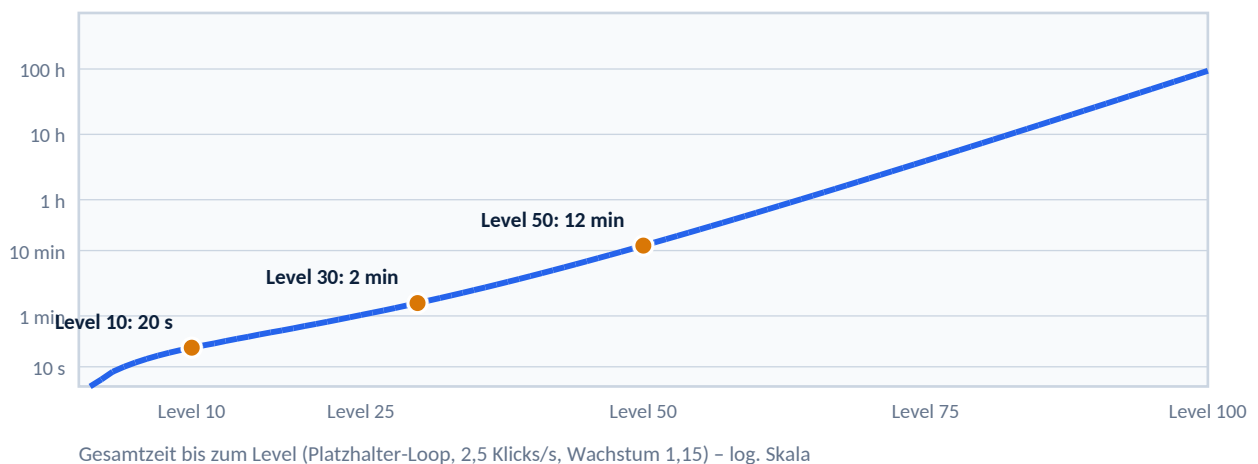


Abb. 13 — Gesamtzeit bis zu einem Level im Platzhalter-Loop (berechnet mit den echten Kit-Formeln).

Wachstum	Level 10	Level 30	Level 50	Gefühl
1,1	17 s	48 s	2 min	großzügig, sehr schnell
1,15	20 s	2 min	12 min	Standard
1,2	22 s	4 min	69 min	zäh, Spieler brauchen Boosts

Berechnet für 2,5 Klicks pro Sekunde, Basispreis 10, Ertrag = 1 + Level. Dein Loop hat andere Zahlen; **die Methode bleibt**: Tabelle in Google Sheets bauen und die Zeit bis zu jedem Meilenstein ausrechnen.

TIPP

Die 4 Regeln der Ökonomie:

1. In den ersten 10 Minuten kommt **alle 1–2 Minuten** ein neues Ziel.
2. Kein Spieler wartet länger als **90 Sekunden ohne sichtbaren Fortschritt**.
3. Boosts (bezahlt) **verkürzen** den Weg, aber ohne Kauf ist das Spiel **voll spielbar**.
4. **Nie rückwirkend nerfen** (Spieler entwerten): Neues Balancing gilt für Neues.

Inflations-Check (wöchentlich): Bleibt die Durchschnitts-Münzenmenge pro Spieler und Stunde stabil? Wächst sie ungebremst, fehlen Senken. Die Kit-Events `LogEconomyEvent` liefern dir die Rohdaten (Quelle/Senke).

5.4 Monetarisierung (Geld verdienen, ohne Spieler zu vergraulen)

Erst Spaß, dann Shop. Der Shop kommt **nach** dem ersten Erfolgserlebnis, nie davor. Verkaufe **Komfort, Tempo und Stil**, nicht Siege.

Typ	Was	Wann sinnvoll	Beispiele
Game Pass	einmalig, für immer	Dauer-Vorteile	2x Münzen, Auto-Sammeln, VIP-Bereich, Extra-Slots
Developer Product	beliebig oft kaufbar	Verbrauchsgüter	Münzpakete, zeitlich begrenzte Boosts, Timer überspringen
Abo (Subscription)	monatlich	Regelmäßiger Mehrwert	VIP-Status mit täglichen Boni (erst nach dem Launch)
Premium-Spieler	Roblox bezahlt Spielzeit von Premium-Nutzern △	automatisch	Kleiner Premium-Vorteil im Spiel als Anreiz

Preisleiter als Startpunkt (Robux, später per Test justieren)

Preis	Passt für	Beispiel
49–99	Einstieg, Impulskauf	Kleines Münzpaket, Namens-Tag
199–249	Dauer-Boost	2x Münzen (Game Pass)
499	Großes Paket	VIP-Bereich + Boost, seltenes Haustier
999+	Sammler/Unterstützer	Supporter-Paket, kosmetische Vitrine

Die Zahlen sind **Startwerte aus meiner Erfahrung, keine Roblox-Regel**. Du testest sie mit A/B (Kapitel 6.4).

```
-- Client.client.lua: Knopf für das Developer Product "500 Münzen"
local productButton = makeButton("Buy500", "500 MÜNZEN", UDim2.fromScale(0.5, 0.28), UDim2.fromScale(0.3, 0.08),
    Color3.fromRGB(147, 51, 234))
productButton.Activated:Connect(function()
    local id = Config.PRODUCTS.Coins500.id
    if id ~= 0 then
        MarketplaceService.PromptProductPurchase(player, id)
    else
        showNotice("Trage zuerst die Produkt-ID in Config ein.")
    end
end)
```

ACHTUNG

Regeln, die Spieler und Plattform schützen:

- **Keine Zufalls-Käufe ohne Quotenangabe** (Kisten, Eier). Roblox verlangt bei bezahlten Zufallsitems Offenlegung der Wahrscheinlichkeiten und schränkt sie regional ein [△](#) (Prüfliste F6). Am einfachsten: **gar keine** bezahlten Zufallsitems.
- **Keine Druck-Tricks auf Kinder** (fake Countdown, «nur noch 1 Minute!», Schuldgefühl-Texte).
- **Keine direkte Kaufaufforderung an Kinder** im Spieltext («Kauf jetzt!»). In Deutschland ist das nach meinem Kenntnisstand wettbewerbsrechtlich heikel [△](#) (UWG Anhang Nr. 28) und wird auf Anwaltsliste für Kapitel 9.6 gesetzt.
- **Keine Kaufpop-ups** in den ersten 3 Minuten, nie mitten in der Aktion.

5.5 Gründe, wiederzukommen (Retention-Mechaniken)

Mechanik	Wirkung	Aufwand	Im Kit?
Tägliche Belohnung + Serie	★★★	klein	ja
Tägliche/Wöchentliche Quests	★★★	mittel	nein
Sammelbuch (Completionist)	★★★	mittel	nein
Timer / Offline-Fortschritt («Ernte ist fertig»)	★★★	mittel	nein
Freunde-Bonus (Spieler mit Freunden verdienen mehr)	★★☆	klein	Beispielcode (unten)
Wochen-Events mit Sonderbelohnung	★★★	mittel	nein
Handeln unter Spielern	★★★	groß (Sicherheit!)	nein
Rebirth/Prestige	★★☆	mittel	nein
Saison-Pass (30 Tage Fortschrittspfad)	★★☆	groß	nein

```
-- Freunde-Bonus: +10 % pro Freund im selben Server (max. +50 %)
local Players = game.GetService("Players")
local friendCache = {} -- [Player] = { [UserId] = true/false }

local function isFriend(player, other)
    friendCache[player] = friendCache[player] or {}
    local cached = friendCache[player][other.UserId]
    if cached ~= nil then
        return cached
    end
    local ok, result = pcall(function()
        return player:IsFriendsWith(other.UserId) -- Web-Aufruf: Ergebnis zwischenspeichern!
    end)
    friendCache[player][other.UserId] = ok and result == true
    return friendCache[player][other.UserId]
end

local function friendMultiplier(player)
    local count = 0
    for _, other in Players:GetPlayers() do
        if other ~= player and isFriend(player, other) then
            count += 1
        end
    end
    return 1 + math.min(count, 5) * 0.1
end
```

ACHTUNG

Handeln zwischen Spielern ist ein starker Haken, aber das Einfallstor für Betrug und Duplikations-Bugs. Baue es erst **nach dem Launch**, mit Server-Prüfung und Protokoll jedes Handels.

5.6 Soziales und Wachstum durch Freunde

- **Co-op-Bonus:** gemeinsam spielen = mehr Belohnung (nie Zwang).
- **Zeigen:** seltene Items sichtbar machen (Aura, Name, Vitrine), das macht neugierig und erzeugt Clips.
- **Einladen:** Roblox-Einladungsfunktion nutzen (Social Prompt), mit Belohnung für beide.
- **Ranglisten** (täglich/wöchentlich), damit auch Neue eine Chance haben.
- **Private Server** für Freundesgruppen (optional, kann Einnahmen bringen).

5.7 Oberfläche und Handy

- ☐ Knöpfe **mindestens 44 × 44 Pixel**; alles mit `Scale (UDim2.fromScale) + UICornerRadius`.
- ☐ Wichtiges **nicht unter den Daumen**-Bereich legen; oben rechts liegt Roblox' eigenes Menü.

- ☐ Texte **groß und kurz**; Zahlen mit K/M/B abkürzen (macht das Kit).
- ☐ Im Studio den **Geräte-Emulator** nutzen (Tab Test → Geräteauswahl, Namen können abweichen) und mindestens ein **echtes Handy** testen.
- ☐ Keine Pflicht-Tastatur; Controller/Gamepad wenigstens grundlegend.
- ☐ Ladezeit: Erstes Bild nach wenigen Sekunden, schwere Inhalte nachladen.

5.8 Performance (damit es auf schwachen Handys läuft)

- ☐ Workspace . StreamingEnabled bei größeren Welten anschalten.
- ☐ Statische Teile **Anchored**; für Deko CanCollide/CanQuery/CanTouch aus.
- ☐ Weniger, größere **MeshParts** statt tausender kleiner Parts; Partikel und Lichter sparsam.
- ☐ Keine schweren Schleifen pro Frame; task.wait() statt wait().
- ☐ Messen: **MicroProfiler (Strg+F6)** und **Developer Console (F9)** (Speicher, Skriptaktivität); in Analytics den Reiter «Performance» ansehen.
- ☐ Teste auf dem **schwächsten** Gerät, das du hast. Stürzt es ab, ist das ein Launch-Blocker.

5.9 Anti-Exploit (Schutz vor Schummeln)

- ☐ Alles, was Geld/Fortschritt betrifft, **nur auf dem Server** (macht das Kit).
- ☐ Jedes RemoteEvent: **Typ, Bereich, ganze Zahl** der Argumente prüfen; **Rate-Limit** (macht das Kit für Collect/Upgrade).
- ☐ Kosten und Belohnungen **serverseitig aus Config**, nie vom Client übernehmen.
- ☐ Admin-Befehle nur serverseitig und nur für eine feste **UserId-Liste**.
- ☐ Nichts Geheimes in ReplicatedStorage oder StarterGui.
- ☐ Auffällige Werte **loggen** (z. B. mehr als X Münzen pro Minute) und Konten prüfen.
- ☐ **Selbsttest**: Schreibe einen kleinen LocalScript, der alle Remotes mit Unsinn feuert (Text statt Zahl, negative, riesige Zahlen). Das Spiel darf nicht abstürzen und nichts gutschreiben.

5.10 Sicherheit für Kinder und Plattform-Regeln

- ☐ **Alters-/Reife-Fragebogen** (Maturity) im Creator Hub **wahrheitsgemäß** ausfüllen. Falsche Angaben können zur Entfernung führen [△](#).
- ☐ Chat nur über Roblox' eigenes, **gefiltertes** Chat-System. Eigene Texteingaben immer über den Roblox-Textfilter schicken (TextService:FilterStringAsync).
- ☐ Seit Jan 2026 ist Chat nur nach **Altersprüfung** (Gesichts-Scan) möglich: plane **chat-unabhängigen** Spielspaß ein (Emotes, Schnell-Nachrichten).
- ☐ **Keine** Aufforderung, Roblox zu verlassen (Discord, Social Media, Messenger) im Spiel.
- ☐ Keine persönlichen Daten erfassen; keine externen Links im Spiel.
- ☐ Melden/Blockieren muss funktionieren (Roblox-Standard nicht abschalten).
- ☐ Orte mit Bar/Club/Schlafzimmer-Charakter nur für 17+ [△](#); vermeide sie.
- ☐ **Discord/TikTok erst ab 13**: richte Community-Kanäle dafür ein, und bewirb sie nicht gezielt an Jüngere.

5.11 Optik, Ton, KI-Werkzeuge

- **Stilguide** (1 Seite): 5 Farben, Formenstil (rund/eckig), Schriftart, Beispiel-Screenshot. Konsistenz schlägt Detail.
- **Low-Poly/stilisiert** ist günstig, hübsch auf Handys und schnell zu produzieren.

- **KI/Studio-Hilfen:** Roblox hat seit 2026 das **Cube**-Modell (3D-/4D-Generierung, Beta) und einen Studio-Assistenten. Nutze sie für Entwürfe, prüfe Qualität, Performance (Dreiecke) und Lizenz.
- **Creator Store:** Modelle/Bilder/Sounds prüfen (Autor, Bewertung, **versteckte Skripte** in Modellen sind ein bekanntes Risiko, lösche unbekannte Scripts).
- **Musik/Sounds:** nur lizenzierte Roblox-Audio-Assets oder selbst erstellt.
- **Freelancer-Auftrag:** Ziel, 3 Stil-Referenzen, Formate/Größen, Deadline, Preis, und **schriftliche Übertragung aller Nutzungsrechte**. Zugriff nur über Gruppenrolle.

5.12 Sprachen

Englisch ist die Hauptsprache der Plattform. Plane **Englisch + Deutsch**. Schreibe **alle Texte in Tabellen/Config** (nie fest im Code), und nutze Roblox' Lokalisierung (Game Settings → Localization, automatische Übersetzung; Bedienung ggf. in der Doku prüfen). Lass die englischen Texte **von einem Muttersprachler gegenlesen**.

MACH JETZT — GATE G-2 und G-3

Gate G-3 (Woche 11): Inhalt für mindestens 2–3 Stunden · alle Shop-Artikel per Testkauf geprüft · Anti-Exploit-Checkliste erledigt · Ø Session im Playtest ≥ 15 Minuten · keine bekannten Datenverlust-Bugs · Fragebogen ausgefüllt.

TESTEN · WOCHE 5–15

Kapitel 6 – Testen, Beta und Messen

Dein Bauchgefühl ist wertvoll, aber **Fremde mit Zahlen** entscheiden. In diesem Kapitel lernst du, wie du Spieltests richtig durchführst, eine geschlossene Beta aufsetzt und die richtigen Zahlen liest.

6.1 Playtest: so läuft ein guter Test

6.1 Der 30-Minuten-Playtest

Wer: 5–10 Menschen **aus deiner Zielgruppe**, die dein Spiel nicht kennen. Kinder nur mit Einverständnis der Eltern, am besten mit Elternteil in der Nähe.

Setup: Sie spielen auf **ihrem** Gerät (Handy!). Du schaust zu oder siehst per Bildschirmteilung zu. Bitte sie, **laut zu denken**.

Deine Regel: 10 Minuten lang **nichts erklären, nicht helfen**. Jede Frage, die gestellt wird, ist ein Fehler im Spiel, nicht beim Tester.

Notieren: Sekunden bis zur ersten Aktion, bis zur ersten Belohnung; Stellen mit Stirnrunzeln; Verwirrung; **wo hört er auf?**

Nachgespräch (3 Fragen): Was war das Beste? Was hat genervt? Was würdest du einem Freund sagen? Und: «Würdest du morgen wieder spielen? Warum (nicht)?»

Beobachtungsbogen	Tester 1	Tester 2	Tester 3
Sekunden bis zur ersten Aktion			
Sekunden bis zur ersten Belohnung			
Stelle der ersten Verwirrung			
Hört auf nach ... Minuten (warum?)			
Hat verstanden, was das Ziel ist? (ja/nein)			
Würde morgen wieder spielen? (ja/nein)			

Auswertung: Zähle die Probleme. Behebe die **3 häufigsten**, nicht die 30. Teste erneut mit **anderen** Menschen (Wiederholer sind voreingenommen).

6.2 Closed Beta aufsetzen (Woche 12)

In der Closed Beta spielen nur Eingeladene. So sammelst du **echte Retention-Daten**, ohne dass das Spiel öffentlich ist. Das Kit hat dafür das Modul **BetaGate** eingebaut (getestet): Wer nicht auf der Liste steht, wird freundlich getrennt.

6.2 Beta-Zugang konfigurieren

1. In Config setzen: `Config.BETA_ONLY = true`.
2. Tester eintragen: `Config.BETA_ALLOWLIST[123456789] = true` (die Zahl ist die **UserId**; Profil-Adresse `roblox.com/users/ZAHL/profile`). Oder `Config.BETA_GROUP_ID` auf deine Gruppe setzen: alle Mitglieder ab `BETA_MIN_RANK` dürfen rein.
3. Das Spiel auf **Öffentlich** stellen (aber **ohne Werbung** und ohne es zu bewerben). Wer nicht auf der Liste steht, kann es zwar öffnen, wird aber sofort getrennt.
4. Tester einladen: Link zum Spiel schicken. Bitte um **Ehrlichkeit**, nicht um Lob.
5. **Zum Launch:** `BETA_ONLY = false` und neu veröffentlichen (nicht vergessen, siehe Launch-Checkliste).

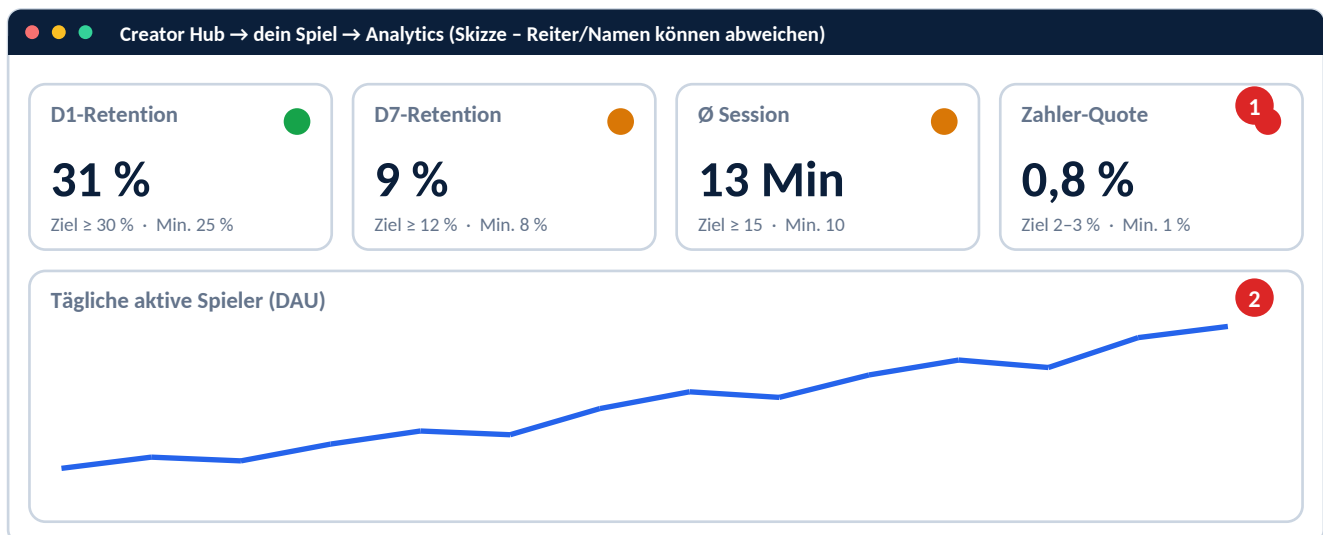
TIPP

Beta-Tester **motivieren**: Ein Beta-Abzeichen (Roblox Badge), ein exklusives Haustier, ein Eintrag in den «Danksagungen». Sammle Rückmeldungen in einem einzigen Ort (Formular oder Discord-Kanal).

6.3 Die Zahlen, die zählen (KPIs)

Kennzahl	Bedeutung	Gute Richtwerte*
DAU	aktive Spieler pro Tag	Trend nach oben, auch ohne Werbung
D1-Retention	Anteil, der am Tag nach dem ersten Besuch wiederkommt	Min. 25 %, Ziel ≥ 30 %
D7-Retention	... nach 7 Tagen	Min. 8 %, Ziel ≥ 12 %
D28-Retention	... nach 28 Tagen (zählt für Discovery!)	Trend beobachten; je höher, desto besser
Ø Session	durchschnittliche Spielzeit pro Besuch	Min. 10 Min, Ziel ≥ 15 Min
Zahler-Quote	Anteil der Spieler, die etwas kaufen	Min. 1 %, Ziel 2–3 %
ARPPDAU	Umsatz pro aktivem Spieler und Tag	Eigenen Wert messen; Modell nutzt 0,05 \$ nur als Annahme
Absturz-/Fehlerrate	Sessions mit Fehler	unter 1 %, Ziel unter 0,5 %

* Meine Daumenregeln aus Benchmarks (z. B. GameAnalytics: Kurzsessions unter 10 % D1) und Praxis. **Keine Roblox-Vorgaben.** Im Creator Hub kannst du dein D1 mit **ähnlichen Spielen** vergleichen: das ist der bessere Maßstab. Ziel: obere Hälfte.



- 1 Ampel pro Kennzahl: **grün** = Ziel erreicht, **gelb** = zwischen Minimum und Ziel, **rot** = unter Minimum. Jeden Montag ausfüllen (Anhang B).
- 2 DAU-Kurve: steigt sie nach einem Update oder nur durch Werbung? Nur Update = gesunde Basis.

Abb. 14 — Dein Montags-Dashboard (Skizze): vier Kennzahlen mit Ampel plus DAU-Kurve. Reiter-Namen im Creator Hub können abweichen.

Wo finde ich das? Creator Hub → dein Spiel → **Analytics** (Reiter wie Engagement, Retention, Monetization, Acquisition, Performance; Namen können abweichen). Die Kit-Ereignisse (Onboarding-Trichter, Economy-Events) erscheinen dort nach einiger Zeit ⚠ (Anzeige ggf. mit Verzögerung).

6.4 Der Verbesserungs-Kreislauf (jede Woche)

1. **Messen:** Montag Dashboard: Wo ist der größte Absprung im Trichter?
2. **Hypothese:** «Wenn ich X ändere, steigt D1 von A auf B, weil ...» (aufschreiben!).
3. **Bauen:** nur **eine** Änderung pro Test (sonst weißt du nicht, was gewirkt hat).
4. **Ausrollen:** an alle oder per A/B (unten).
5. **Lernen:** nach 7 Tagen: Hat sich die Zahl bewegt? Ja → behalten. Nein → zurück, nächste Hypothese.

A/B-Test im Spiel: Weise Spieler dauerhaft einer Gruppe zu (gleiche UserId = gleiche Gruppe) und logge die Gruppe als Ereignis. Beispiel (die Funktion gehört in Main oder ein eigenes Modul):

```
-- Einfacher A/B-Test: Spieler dauerhaft einer Gruppe zuordnen (gleiche UserId = gleiche Gruppe)
local function variantOf(player, testName)
    local hash = (player.UserId + #testName * 7919) % 2
    return if hash == 0 then "A" else "B"
end

-- Beispiel: Startgeschenk testen (A = 50 Münzen, B = 100 Münzen)
DataService.onLoaded(function(player, data)
    if data.totalCollected == 0 then
        local variant = variantOf(player, "startgift")
        EconomyService.addCoins(player, if variant == "A" then 50 else 100, "startgift_" .. variant)
        Analytics.custom(player, "startgift_" .. variant, 1)
    end
end)

-- Danach: D1/D7 der beiden Gruppen im Dashboard vergleichen. Immer nur EINE Änderung pro Test.
```

Außerdem kannst du **Thumbnails/Icons** im Creator Hub testen lassen (A/B-Test der Store-Bilder; Menü im Creator Hub nachsehen ⚠). Das ist der günstigste Hebel für mehr Besucher.

6.5 Diagnose: Was tun bei schlechten Zahlen?

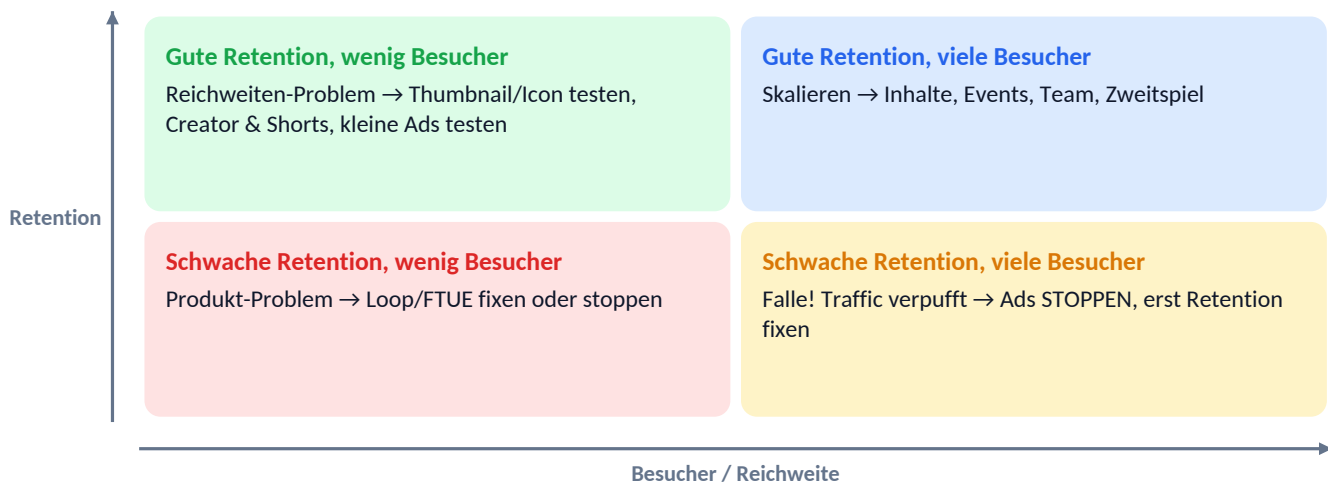


Abb. 15 — Die Vier-Felder-Diagnose: Retention und Reichweite getrennt betrachten.

6.6 Beta-Technik: der Härtestest

- ☐ **100-mal Beitreten/Verlassen** (z. B. mit Test → Clients and Servers) ohne Datenverlust.
- ☐ **Mehrspieler:** ≥ 8 Clients gleichzeitig, kein Lag-Anstieg, keine Fehler im Server-Log (F9 → Server-Log).
- ☐ **DataStore-Ausfall simulieren** (Internet am PC kappen oder API-Schalter aus): Das Spiel darf niemanden mit leeren Daten starten lassen (macht das Kit).
- ☐ **Restore-Plan:** Wie stellst du einen Spielstand wieder her? (Datensicherung des DataStore-Eintrags, Ablauf aufschreiben.)
- ☐ Auf **3 verschiedenen Geräten** spielen: Low-End-Handy, Tablet, PC.
- ☐ Alle **Käufe** einmal mit Testkonto durchführen und die Gutschrift prüfen.

6.7 Beta-Umfrage (nach Tag 2 und Tag 7)

- Wie hat dir das Spiel gefallen? (1–5)
- Was war dein Lieblingsmoment?
- Was war verwirrend oder nervig?
- Wie wahrscheinlich ist es, dass du es Freunden empfiehlst? (0–10)
- Was fehlt dir? (Freitext)
- Würdest du für ... Geld ausgeben? Was würdest du kaufen?

6.8 Die Gates G-2 und G-4 (Ampel-Entscheidung)

Metrik	Mindestens	Ziel	Gate
Spaß-Test (G-2): freiwillig ≥ 10 Min gespielt	7 von 10	9 von 10	Woche 6
Spaß-Test: «würde wieder spielen»	50 %	70 %	Woche 6
Beta (G-4): D1-Retention	25 %	≥ 30 %	Woche 15
Beta: D7-Retention	8 %	≥ 12 %	Woche 15
Beta: Ø Session	10 Min	≥ 15 Min	Woche 15
Beta: Zahler-Quote	1 %	2–3 %	Woche 15
Beta: Fehler-/Absturzrate	unter 1 %	unter 0,5 %	Woche 15

ACHTUNG — KILL-REGELN

Stopp- und Pivot-Regeln (vorher festlegen!):

- G-2 nach 2 Iterationen verfehlt → Pivot oder Stopp. Prototyp-Kosten sind klein.
- G-4: nach zwei zusätzlichen 2-Wochen-Zyklen immer noch $D1 < 15$ % oder $D7 < 3$ % → Pivot oder **Stopp**.
- Gute Retention, aber zu wenig Zahler → Monetarisierung überarbeiten, nicht den Loop.

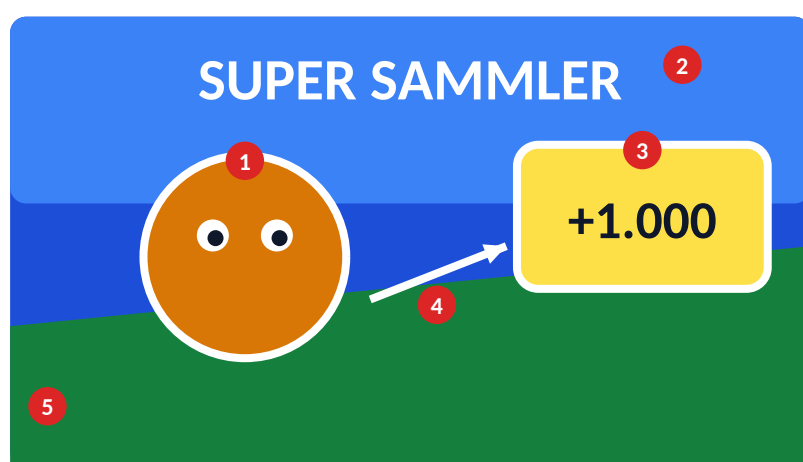
LAUNCH · WOCHE 16–19

Kapitel 7 – Launch: Store-Seite, Marketing, Start

Ein gutes Spiel, das niemand findet, hilft keinem. Der Launch besteht aus **drei Teilen**: einer Store-Seite, die klickt, Inhalten, die neugierig machen, und einem Start-Tag, an dem alles gleichzeitig passiert.

7.1 Die Store-Seite (Icon, Thumbnails, Titel)

Spieler entscheiden in ca. **1 Sekunde** auf einem kleinen Bild, ob sie klicken. Das Thumbnail ist deine wichtigste Werbung.



- 1 Ein klares Hauptmotiv, groß, Gesicht/Emotion.
- 2 Max. 2–3 Wörter Text, riesig, hoher Kontrast.
- 3 Belohnung zeigen (Zahl, Item) – das Versprechen.
- 4 Blickführung: Pfeil/Linie vom Motiv zur Belohnung.
- 5 Kontrast Vorder- zu Hintergrund: auf dem Handy-Mini-Format testen!

Nie täuschen: Keine Bilder von Dingen, die es im Spiel nicht gibt – das verstößt gegen die Regeln und zerstört die Retention.

Abb. 16 — Anatomie eines klickenden Thumbnails (Beispiel, selbst gezeichnet).

Element	Regel
Icon	Quadratisch (laut Doku 512 × 512 △), ein Motiv, auch klein erkennbar.
Thumbnails	16:9 (laut Doku 1920 × 1080 △), 3–5 Stück: (1) Hauptbild, (2) Aktion, (3) Belohnung/Seltenes, (4) Freunde/Multiplayer. Teste 3 Varianten (A/B im Creator Hub).
Titel	Kurz, merkbar, mit einem Suchbegriff (was ist es?). Keine Sonderzeichen-Orgie, keine Titel, die Marken nachahmen.
Beschreibung	Zeile 1 = Versprechen. Danach 4–6 Stichpunkte (Features), Codes, Update-Rhythmus. Englisch zuerst, Deutsch ergänzen.
Genre/Tags	Das passende Genre wählen; Maturity-Einstufung ehrlich.
Social-Links	Nur über die dafür vorgesehene Funktion im Creator Hub, und nur altersgerechte Kanäle.

Beschreibung (Vorlage):

[SPIELNAME] – Sammle, verbessere, zeige es deinen Freunden!

- * Starte in 10 Sekunden – ohne Anleitung
- * Finde seltene [ITEMS] und baue deine Sammlung auf
- * Spiele mit Freunden: gemeinsam gibt es Bonus-Belohnungen
- * Neue Updates JEDE Woche, neue Events jeden Monat
- * Code: LAUNCH (löse ihn im Spiel ein)

7.2 Kurzvideos, die neugierig machen

TikTok, YouTube Shorts und Instagram Reels (alle erst ab 13) sind dein günstigster Marketing-Kanal.

Gameplay-Previews mit gängigem Audio schlagen polierte Trailer.

Sekunde	Was passiert	Beispiel
0–1	Hook: stärkster Moment sofort	Die seltenste Belohnung, ein riesiger Zahlensprung
1–5	Kernaktion zeigen	Wie man es spielt, ohne Erklärtext
5–15	Fortschritt im Zeitraffer	Level 1 → Level 30 in 10 Sekunden
15–25	«Wow» oder Social	Freunde, Event, selten, lustig
25–30	Aufruf + Name	«Spiele [SPIEL]. Link in Bio.»

- **Hochformat 9:16**, Text-Einblendung in den ersten 2 Sekunden, Untertitel.
- **Rhythmus:** ab Woche 6 zwei Clips pro Woche («Devlog»), zum Launch 15–20 Clips auf Vorrat.
- **Ideen-Fundus:** Vorher/Nachher, «Level 1 gegen Level 100», seltenes Item, lustige Fails, Wochen-Event-Teaser, Community-Highlights, «Ich baue ein Roblox-Spiel – Tag 17» (Entwickler-Tagebuch).
- **Nutze nur lizenzierte Musik** (aus der Plattform-Bibliothek).

7.3 Creator-Zusammenarbeit

1. Liste **30 kleine bis mittlere Creator** (10.000–500.000 Abos), die Roblox-Spiele deiner Art zeigen. Mega-Creator (Millionen) sind für dich unerreichbar und unnötig.
2. Schau ihre letzten Videos: Zeigen sie **neue Spiele**? Wie sind Kommentare? Ist die Zielgruppe passend?
3. **Pitch kurz, persönlich, mit Link** zum Early-Access (Vorlage unten). Nie mehr als 3 Sätze Selbstlob.
4. **Konditionen:** kostenloser Early-Access, exklusive Items/Codes, bei größeren Deals Umsatzbeteiligung (5–15 % im ersten Monat laut einer Quelle [△](#), verhandelbar). **Immer schriftlich.**
5. Gib jedem Creator einen **eigenen Code** (z. B. MAX): Belohnung für Spieler und Auswertung, wer wie viele bringt.

Betreff: Early-Access: [SPIELNAME] (neues Roblox-Spiel) für [KANALNAME]

Hi [Name],
 ich habe dein Video «[Titel]» gesehen und glaube, mein neues Roblox-Spiel «[SPIELNAME]»
 passt gut zu deinen Zuschauern: [1-Satz-Loop]. Hier ist der Early-Access-Link: [LINK].
 Dein Code «[CODE]» gibt deinen Zuschauern [Belohnung].
 Wenn du es zeigen möchtest: Launch ist am [DATUM]. Ich schicke dir gern Clips/Assets.
 Danke dir! [Name]

(English: Hi [Name], I saw your video "[Title]" and think my new Roblox game "[GAME]" fits your
 audience: [one-sentence loop]. Early-access link: [LINK]. Your code "[CODE]" gives viewers
 [reward]. Launch is on [DATE]. Happy to send clips/assets. Thanks! [Name])

7.4 Community (Discord ab 13)

- **Kanäle:** #willkommen (Regeln), #news, #updates, #feedback, #bugs, #clips, #off-topic. Wenige Kanäle sind besser als viele leere.
- **Regeln:** freundlich, keine Werbung, keine persönlichen Daten, keine Kontaktanfragen an Jüngere. Mindestens 2 **Moderatoren**.
- **Rhythmus:** Patchnotes jeden Freitag, Roadmap-Teaser jeden Sonntag, Umfragen vor großen Entscheidungen.
- Plattform-Altersgrenzen (Discord/TikTok ab 13) beachten: bewirb diese Kanäle **nicht** gezielt an Jüngere.

7.5 Werbung (Roblox Ads): erst rechnen, dann kaufen

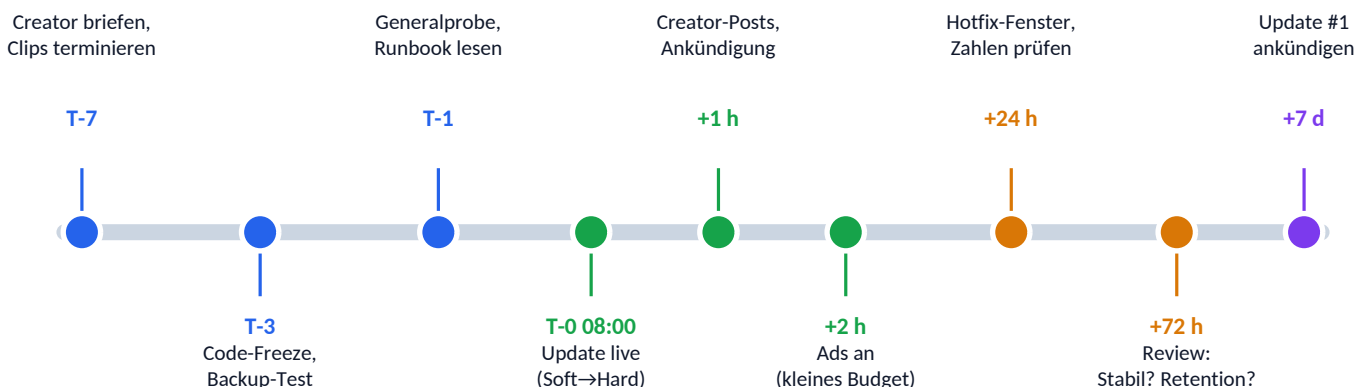
Roblox bietet bezahlte Werbung für Spiele (Creator Hub → Anzeigen/Ads; Name im Hub prüfen △). **Die Regel:** Gib nie mehr aus, als ein neuer Spieler in 30 Tagen einbringt.

Aktive Tage in 30 Tagen $\approx 1 + \text{Summe der täglichen Retention}$
 (Beispiel D1 30 %, D7 12 %, D28 5 % → ca. 4,3 Tage)
 Wert eines neuen Spielers (30 Tage) = aktive Tage $\times$ ARPDAU $\times$ Entwickleranteil
 $= 4,3 \times 0,05 \$ \times 0,22 \approx 0,05 \$$
 → Du darfst höchstens ca. 0,05 \$ pro neuem Spieler bezahlen.

ACHTUNG

Mit den Beispielzahlen lohnt sich Werbung **nur**, wenn Neuspeler sehr billig sind **oder** dein ARPDAU höher ist. Fange mit **kleinem Testbudget (300–500 €)** erst nach dem Soft Launch an, miss die echten Kosten pro Spieler und stoppe, wenn die Rechnung nicht aufgeht.

7.6 Der Launch-Plan (Runbook)



Alle Zeiten sind Vorschläge; Wichtig: Creator-Posts und Update liegen am SELBEN Tag.

Abb. 17 — Von T-7 Tage bis +7 Tage nach dem Launch (Vorschlag).

Vor dem Launch: Betriebshandbuch aufschreiben

- ☐ **Überwachung:** Wer schaut wann auf CCU, Fehlerrate (F9 → Server-Log), Käufe, Discord-Meldungen? Schichtplan für die ersten 72 Stunden.
- ☐ **Hotfix-Weg:** Änderung im Studio → Test → Publish. Bei Notfällen: **Server neu starten** (Option im Creator Hub, Name prüfen △), damit alte Server die neue Version bekommen.
- ☐ **Rollback:** Version im Versionsverlauf wiederherstellen (Creator Hub). Einmal in der Beta geübt?
- ☐ **Notfallschalter:** Ein Config-Schalter, der Shop oder ein Feature abschaltet (Config ändern → veröffentlichen → Server neu starten).
- ☐ **Datensicherung:** Wie holst du einen Spielstand zurück? Ablauf aufschreiben.

☐ **Eskalation:** Wen rufst du bei Datenverlust, Exploit oder Kaufproblemen? (Eine Person + Stellvertretung.)

7.7 Soft Launch, dann Hard Launch

Quellen widersprechen sich (leise starten vs. am Starttag pushen). **Meine Empfehlung: beides, nacheinander.** Eine Woche **öffentlich, aber ohne Werbung** zum Stabilisieren (Soft Launch, Woche 18), dann ein **konzentrierter** Push an einem festen Tag (Hard Launch, Montag Woche 19).

Gate G-6 (Ende Soft Launch)	Bedingung
Stabilität	keine P0-Bugs (Datenverlust, Käufe, Abstürze)
Retention	D1 ≥ 80 % des Beta-Werts
Wirtschaft	keine Exploits/Duplikationen; Münzen pro Spieler/Stunde plausibel
Betrieb	Server-/DataStore-Warnungen im grünen Bereich; Runbook von einer zweiten Person gegengelesen

MACH JETZT — START

Launch-Tag (Montag, Woche 19): Beta-Schalter AUS · Update live · Creator-Posts **am selben Tag** · Ankündigung in Discord · Werbung an (klein) · 72-Stunden-Bereitschaft. Detaillierte Checkliste: Anhang B.

7.8 Letzte Prüfung (Recht und Plattform)

- ☐ Fragebogen (Maturity/Alter) aktuell und wahrheitsgemäß.
- ☐ **Impressum + Datenschutz** auf Website/Link-in-Bio, wenn du geschäftsmäßig Social-Kanäle betreibst (Impressumspflicht prüfen) [△](#).
- ☐ Alle Assets lizenziert (Musik, Bilder, Modelle). Keine Marken im Namen.
- ☐ Keine direkte Kaufaufforderung an Kinder im Spiel (siehe 5.4).
- ☐ Steuerliche Erfassung mit Steuerberater geklärt (Kapitel 9.6).

LIVE-OPS · AB WOCHE 20

Kapitel 8 – Live-Betrieb und Wachstum

Erst im Betrieb entscheidet sich, ob dein Spiel wächst. Der Rhythmus aus **messen, verbessern, ausliefern** ist alles. Plane diese Phase wie einen zweiten Job.

8.1 Der Wochenrhythmus (fester Kalender)

Tag	Aufgabe	Ergebnis
Mo	KPI-Review (Anhang B): D1/D7/D28, Session, Zahler-Quote, DAU, Fehler. Hypothese festlegen.	Wochenziel
Di-Mi	Entwickeln: kleines Update (neue Items/Quests) + ein Test (A/B).	Build
Do	Test (Mehrspieler, Handy), Patchnotes schreiben, Clips vorbereiten.	Release-Kandidat
Fr	Release (vor dem Wochenende: viele Spieler online). Danach 2 Stunden Beobachtung.	Update live
Sa-So	Community-Post, Roadmap-Teaser, Support. Nichts Riskantes veröffentlichen.	Bindung

8.2 Event-Kalender 2027 (Vorschlag)

Zeitraum	Anlass	Was du tust
Feb 2027	Hard Launch	Launch-Event + Launch-Item (nur kurz verfügbar)
Mär 2027	Frühling, Ostern (So 28.03.)	Oster-Event: Sammel-Eier, Themen-Items (keine Echtgeld-Zufallskisten!)
Mai-Jun 2027	Sommer-Start	Zone/Update «Sommer», Ferien-Boost vor Schulferien
Jul-Aug 2027	Sommerferien (Haupt-Traffic)	Wöchentliche Mini-Events, Creator-Aktion
Okt 2027	Halloween (So 31.10.)	Halloween-Event, Themenwelt (ca. 6 Wochen vorher planen)
Nov 2027	Black Friday (26.11.)	Rabatt-Event (Preis kurz senken, ehrlich kommunizieren)
Dez 2027	Winter/Weihnachten	Winter-Event, Jahresrückblick, Dank an die Community
Feb 2028	1. Geburtstag (15.02.)	Jubiläums-Event, Feier-Item

Plane Events **6 Wochen im Voraus**: Woche 1–2 Entwurf, 3–4 Bau, 5 Test, 6 Teaser. Schulferien-Termine unterscheiden sich je Land: **Wichtig ist die Zielgruppen-Ferienzeit**, nicht nur Deutschland.

8.3 Update-Ideen nach Aufwand

Aufwand	Beispiele	Wirkung
Klein (1–2 Tage)	Neue Items, Haustiere, Quests, Codes, Balance-Fixes, kleine Events	Bindung, Gesprächsstoff
Mittel (1 Woche)	Neue Zone, neues System (z. B. Sammelbuch), Wochen-Event	Reaktivierung, neue Clips
Groß (3–6 Wochen)	Saison, großes Feature, neue Spielart	Neue Spieler, Presse, Creator-Buzz

8.4 Wachstums-Hebel (wenn die Zahlen stocken)

Symptom	Wahrscheinliche Ursache	Gegenmaßnahme
Wenig Besucher, gute Retention	Sichtbarkeit	Thumbnail/Icon testen, Titel, Creator, Shorts, kleine Ads-Tests
Viele Besucher, schlechte D1	FTUE/Loop schwach	Funnel prüfen, erste 60 s straffen, Ladezeit
D1 gut, D7 schlecht	Kein Grund für Wiederkehr	Dailies/Quests, Events, Sammelbuch, Social-Bindung
Gute D7, wenig Umsatz	Shop zu schwach/unsichtbar	Preisleiter prüfen, Shop-Zeitpunkt, Angebote an Erfolgsmomenten
Viel Umsatz, viele Beschwerden	Pay-to-Win-Gefühl	Käufe auf Komfort/Optik zurückführen
Exploits/Schummler	Server vertraut Client	Logs, Limits, Validierung, Ban-Prozess

8.5 Community und Moderation

- **Antwortzeit:** Bug-Meldungen in 24 Stunden bestätigen. Danksagungen in den Patchnotes.
- **Ton:** ehrlich, freundlich, kurze Sätze; Fehler offen zugeben.
- **Moderation:** Regeln sichtbar, Verwarnung vor Bann, bei Belästigung/Kindeswohl sofort Roblox melden. Mindestens 2 Moderatoren, nie allein.
- **Feedback sammeln:** ein Ort (Formular/Kanal), monatliche Umfrage, öffentliche Roadmap.

8.6 Team und Portfolio

- **Erst wachsen, wenn es sich trägt:** Eine Rolle einstellen, wenn der **durchschnittliche Monatsüberschuss der letzten 3 Monate** sie bezahlt. Reihenfolge: Scripter → Builder/Artist → Community-Manager.
- **Verträge schriftlich:** Aufgaben, Preis oder Umsatzbeteiligung, Rechte-Übertragung, Gruppenrolle statt Passwort.
- **Portfolio-Denken (ab ca. Tag 60):** Technik (Daten, Shop, Analytics, Anti-Exploit) in eine **Vorlage** auslagern und ein zweites Experiment in 6–8 Wochen starten. Der Median verdient wenig, mehrere Schüsse erhöhen die Chance auf einen Treffer.

8.7 Dich selbst schützen

- **Festes Zeitbudget** pro Woche; ein freier Tag. Spiele leben Jahre, nicht Wochen.
- **Nie Updates an Freitagabend** ohne Bereitschaft; nie am Wochenende allein riskante Änderungen.
- **Pausenplan:** Wenn du in den Urlaub willst, plane Events und Updates vorher (Vorrat von 2–3 kleinen Updates).

8.8 Reviews nach 30, 60, 90 Tagen

Zeitpunkt	Fragen	Entscheidung
30 Tage (≈ 17.03.2027)	Ist die Retention stabil? Wächst DAU ohne Werbung? Rechnet sich Werbung?	Reichweiten- oder Produktproblem (Kapitel 6.5)
60 Tage (≈ 16.04.2027)	Trägt der Umsatz die laufenden Kosten? Funktioniert das Update-Tempo?	Skalieren, halten, Pivot; zweites Experiment starten?
90 Tage (≈ 16.05.2027)	Break-even erreicht oder in Sicht? Zeit/Nutzen für dich?	Fortsetzen · Skalieren · Wartungsmodus · Stopp

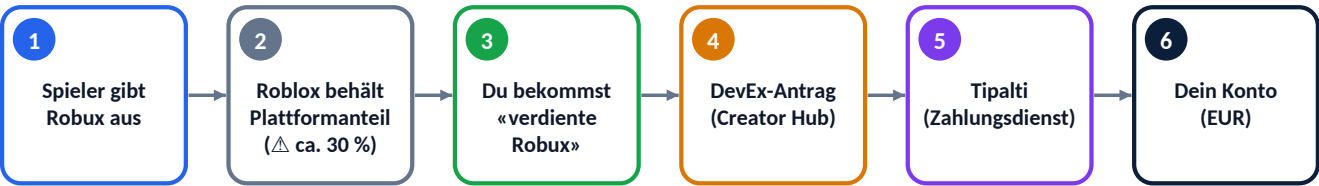
GESCHÄFT

Kapitel 9 – Geld, Steuern und Recht

ACHTUNG

Dieses Kapitel ist **keine Steuer- oder Rechtsberatung**. Es erklärt, wie das Geld fließt, rechnet Beispiele durch und gibt dir die richtigen Fragen für Steuerberater:in und Anwalt/Anwältin mit. Alle Δ -Werte musst du im Creator Hub bestätigen.

9.1 Wie aus Spieler-Robux echtes Geld wird



Was unterwegs abgeht (prüfen, siehe Kapitel 9):

- DevEx-Kurs pro Robux (Quellen widersprechen sich: 0,0035 \$ / 0,0038 \$ / 0,0054 \$ für US-Erwachsene) • Mindestmenge zum Auszahlen
- Tipalti-Gebühr je nach Zahlungsmethode • Währungsumrechnung ca. 1,9–3 % • Steuer-Formular (W-8BEN) vorher hinterlegen

Abb. 18 — Der Weg des Geldes. Jeder Schritt hat Abzüge oder Bedingungen.

9.2 Auszahlung (DevEx) Schritt für Schritt

9.2

So kommst du an dein Geld

- Voraussetzungen prüfen:** Creator Hub → Developer Exchange (DevEx): Alter, verifiziertes Konto, Mindestmenge verdienter Robux Δ (30.000 oder 100.000? Quellen widersprechen sich).
- Steuerformular** abgeben: als Nicht-US-Person **W-8BEN** (über Tipalti; ab 01.11.2026 wird die Steuerinfo in den Creator Hub («Taxes»-Seite) verlagert, Auszahlung bleibt über Tipalti).
- Auszahlungsweg** in Tipalti wählen (z. B. Banküberweisung/PayPal). Jede Methode hat eine **Gebühr**; bei Umrechnung in EUR fällt eine **Währungsgebühr von ca. 1,9–3 %** an.
- Im Creator Hub **DevEx beantragen**. Bearbeitung dauert; plane Wochen, nicht Tage.
- Auszahlen in sinnvollen Tranchen** (nicht jede Woche kleine Beträge) und **Rücklage für Steuern** bilden.

Rechenbeispiel (mit dem vorsichtigen Kurs 0,0035 \$/Robux)

Schritt	Rechnung	Ergebnis
Verdiente Robux	100.000 Robux	100.000
Kurs	$100.000 \times 0,0035 \$$	350 \$
Währungsgebühr ca. 3 % (Worst Case)	$350 \$ \times 0,97$	$\approx 339,50 \$$
Tipalti-Transaktionsgebühr	je nach Methode Δ	abziehen
Steuer (DE)	mit Steuerberater klären	Rücklage bilden

PRÜFEN

Kurs unklar: Quellen nennen **0,0035 \$** (verbreitet), **0,0038 \$** und **0,0054 \$** für Robux von US-Erwachsenen seit Juni 2026. Keine offizielle Bestätigung gefunden. Rechne **konservativ mit 0,0035**, bis du im Creator Hub den echten Wert siehst.

9.3 Wie viel kommt von einem Euro an?

Vereinfacht: Von dem, was ein Spieler ausgibt, behält Roblox etwa 30 % (Plattformanteil $\triangle$). Dazu kommt der Unterschied zwischen dem **Preis**, zu dem Spieler Robux kaufen, und dem **Kurs**, zu dem du sie auszahlen lässt:

$$\begin{aligned}\text{Entwickleranteil} &\approx 0,70 \times \text{DevEx-Kurs} \div \text{Robux-Kaufpreis} \\ &\approx 0,70 \times 0,0035 \div (0,010 \dots 0,0125) \approx 20 \dots 25 \%\end{aligned}$$

→ Von 1 € Spielerausgabe bleiben grob 20–25 Cent bei dir (vor Gebühren und Steuern).

9.4 Szenarien: was ist realistisch?

Die Rechnung verwendet **Annahmen, keine Fakten**: ARPDAU 0,05 \$ (Umsatz pro aktivem Spieler und Tag), Entwickleranteil 22 %, also **0,011 \$ pro DAU und Tag** $\approx$ **4 \$ pro DAU und Jahr**. Ersetze 0,05 \$ so schnell wie möglich durch deinen echten Beta-Wert.

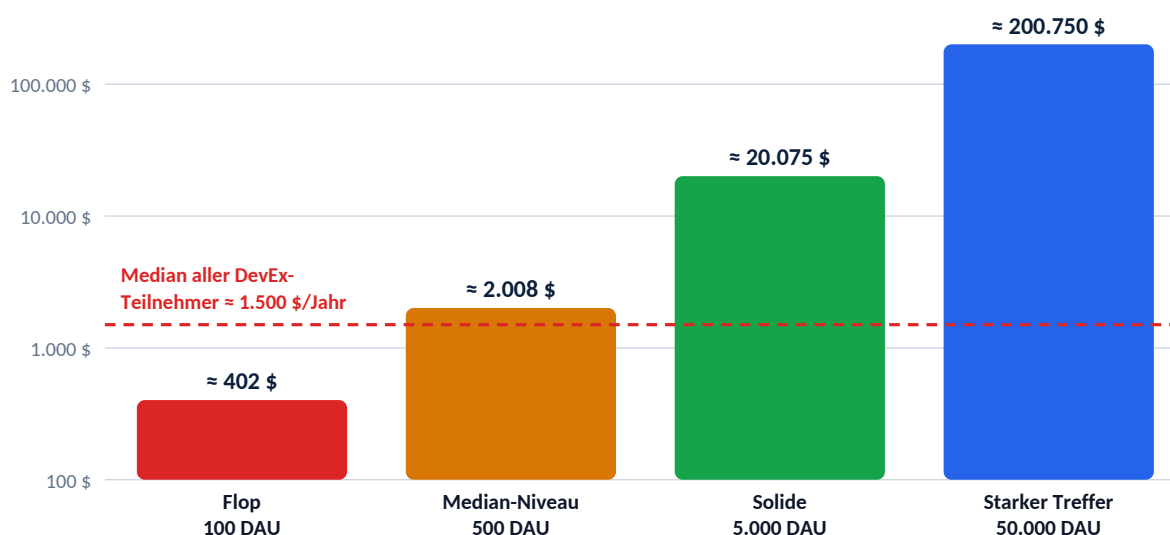


Abb. 19 — Grobe Jahreserlöse bei verschiedenen durchschnittlichen DAU (logarithmische Skala).

Szenario	Ø DAU	Netto-Erlös / Jahr	Einordnung
Flop	100	≈ 400 \$	unter dem Median
Median-Niveau	500	≈ 2.000 \$	etwa Median (≈ 1.500 \$)
Solide	5.000	≈ 20.000 \$	trägt Nebeneinkommen/Freelancer
Starker Treffer	50.000	≈ 200.000 \$	selten

Break-even-Beispiel: Bei 3.000 \$ Gesamtkosten in 6 Monaten brauchst du im Schnitt **ca. 1.500 DAU** ($3.000 \div (0,011 \times 182)$). Die Creator Rewards (5 Robux pro qualifiziertem Spieler-Tag $\triangle$) sind **nicht** eingerechnet.

9.5 Budget-Stufen (eigene Schätzungen)

Posten	Lean	Standard (Plan)	Aggressiv
Werkzeuge/Abos	0–100 €	100–300 €	300 €
Freelancer (UI, Icon, Thumbnails, Sound, Builder)	0 €	800–1.800 €	4.000–8.000 €
Roblox Ads / Creator-Deals	0–100 €	300–800 €	3.000–6.000 €
Steuerberater/Recht (Einrichtung)	150–300 €	200–400 €	400–800 €
Summe	≈ 150–500 €	≈ 1.500–3.000 €	≈ 8.000–15.000 €

9.6 Steuern und Recht in Deutschland: deine Fragenliste

Buche in **Woche 1** ein Erstgespräch (ca. 60 Minuten). Bringe diese Fragen und Unterlagen mit:

- ☐ Ab wann liegt **Gewinnerzielungsabsicht** vor? Brauche ich ein **Gewerbe** oder bin ich freiberuflich eingestuft?
Fragebogen zur steuerlichen Erfassung (ELSTER) wann und wie?
- ☐ Wie behandle ich **Einnahmen aus den USA (DevEx über Tipalti)** in EUR (Umrechnung, Zuflusszeitpunkt)?
- ☐ **US-Quellensteuer** und **W-8BEN**/Doppelbesteuerungsabkommen DE–USA: Muss/kann ich etwas anrechnen?
- ☐ **Umsatzsteuer**: Wie sind Leistungen an eine US-Plattform zu behandeln? Kleinunternehmerregel sinnvoll?
- ☐ **Betriebsausgaben**: Software, Freelancer, Werbung, Arbeitsplatz. Welche Belege brauche ich?
- ☐ **Rechtsform**: Ab welchem Gewinn lohnt sich UG/GmbH? Haftung bei Kinderschutz-/Datenschutz-Vorfällen?
- ☐ **Verträge** mit Freelancern (Rechte-Übertragung) und Creatorn (Umsatzbeteiligung).
- ☐ **Werbung und Kinder**: In Deutschland ist die unmittelbare Aufforderung an Kinder, Waren zu kaufen, wettbewerbsrechtlich problematisch (UWG, Anhang Nr. 28) △: Wie formuliere ich Shop-Texte sicher?
- ☐ **Impressum/Datenschutz** für Website und geschäftsmäßige Social-Accounts (DDG, DSGVO).

9.7 Plattform-Regeln im Blick behalten

- Roblox ändert Auszahlungen, Rewards, Altersregeln und Oberfläche mehrmals im Jahr. **Quartalsweise** Creator Hub, DevForum-Ankündigungen und Newsroom lesen.
- **Nie vom einem Programm abhängig machen** (z. B. Creator Rewards): Plane Einnahmen aus Käufen als Hauptsäule.
- **Meine Hinweis-Liste** offener Fakten steht in 9.8.

9.8 Prüfliste: Das konnte ich nicht bestätigen

Der Creator Hub war aus meiner Umgebung gesperrt. Diese Punkte musst du dort selbst nachsehen und hier ankreuzen, **bevor du Geld aus gibst**.

	Frage	Was Quellen sagen	Wo prüfen
☐ F1	DevEx-Kurs	0,0035 / 0,0038 \$; 0,0054 \$ für US-Erwachsene ab 06/2026 (nicht offiziell bestätigt)	Creator Hub → DevEx
☐ F2	Mindestauszahlung	30.000 oder 100.000 verdiente Robux	Creator Hub → DevEx
☐ F3	DevEx-Voraussetzungen	Alter, ID-Verifizierung, evtl. Premium? Gruppen-Auszahlung?	Creator Hub
☐ F4	Plattformanteil auf Käufe	gängig 30 %, nicht neu bestätigt	Creator Hub → Monetarisierung

	Frage	Was Quellen sagen	Wo prüfen
☐ F5	Pflichten für Kinder-Zielgruppen	Review-Stufen, Plus-Abo? (nur Sekundärquellen)	Creator Hub, DevForum
☐ F6	Bezahlte Zufallsitems	Quoten-Offenlegung, regionale Einschränkungen	Roblox-Richtlinien
☐ F7	Creator Rewards (5 Robux/Spieler/Tag u. a.)	nur aus Zusammenfassungen; Werbeanteil-Änderung ab 01/2027 gemeldet	Creator Hub, DevForum
☐ F8	Studio-Käufe: kosten sie Robux?	ungeprüft	Creator-Hub-Doku
☐ F9	Icon-/Thumbnail-Größen	512×512 / 1920×1080	Creator-Hub-Doku

ZUM ABHAKEN

Kapitel 10 – Dein 19-Wochen-Fahrplan

Eine Zeile pro Woche. Stunden sind Richtwerte bei 15–20 h/Woche. Wenn du hinterherhinkst: **streiche Inhalt, nie Tests.**

Woche	ab	Thema	Aufgaben	Gate	Erledigt
1	12.10.	Fundament	Konto sichern, Studio, Baseplate veröffentlichen, API-Schalter, Gruppe; Steuerberater buchen; Prüfliste F1–F9	G-0	☐
2	19.10.	Idee prüfen	10-Fragen-Check, Marktscan (3–5 Spiele), Spielsteckbrief, Papier-Test		☐
3	26.10.	Kit einbauen	Steckbrief fertig → G-1 ; Starter-Kit einbauen (Kap. 4), erster F5-Test, Mehrspieler-Test	G-1	☐
4	02.11.	Dein Loop (Graybox)	Platzhalter-Loop durch deinen ersetzen, Zahlen in Config, FTUE-Hinweise, erste Belohnung < 60 s		☐
5	09.11.	Daten & Test	Analytics-Ereignisse prüfen, 100× Join/Leave, Playtest #1 mit 10 Personen		☐
6	16.11.	Iteration + Devlog	Top-3-Probleme beheben, Devlog-Kanal starten (2 Clips/Woche) → G-2	G-2	☐
7–8	23.11.	Inhalt (Zone 1–2)	Items, Upgrades, Welten; Ökonomie-Tabelle; Sammelbuch/Quests starten		☐
9	07.12.	Shop	Pass + Produkte anlegen, Preise, Shop-UI, Testkäufe, Kaufzeitpunkte festlegen		☐
10	14.12.	Bindung & Schutz	Daily/Streak/Quests, Anti-Exploit-Review, Mobile-Performance, Safety-Fragebogen		☐
11	21.12.	Politur & Test	Playtest #2/#3, Lokalisierung EN/DE, Bugfix; Weihnachtspause light → G-3	G-3	☐
12	28.12.	Closed Beta Start	BetaGate an, 100–300 Tester, tägliches Monitoring		☐
13–14	04.01.	Beta-Iteration	Wöchentlicher Zyklus (Messen→Hypothese→Bauen→Lernen), Thumbnails testen, Umfrage		☐
15	18.01.	Beta-Abschluss	Retention-Gate prüfen, Entscheidung Go/Iterieren/Stopp → G-4	G-4	☐
16	25.01.	Store & Content	Store-Seite, Thumbnails, Trailer, 15–20 Clips, Creator-Liste (30), Codes		☐
17	01.02.	Outreach & Runbook	Creator-Pitches, Discord, Launch-Runbook, Generalprobe → G-5	G-5	☐
18	08.02.	Soft Launch	Öffentlich ohne Werbung, täglich Logs/Wirtschaft prüfen → G-6	G-6	☐
19	15.02.	HARD LAUNCH	Beta aus, Update live, Creator-Posts, Ads (klein), 72 h Bereitschaft, Update #1 nach 7 Tagen		☐
20+	22.02.	Live-Ops	Wochenrhythmus (Kap. 8), Reviews nach 30/60/90 Tagen		☐

Wenn du in Verzug gerätst

Situation	Reaktion
1 Woche Verzug	Puffer (Weihnachten) nutzen; Inhalt kürzen (eine Zone weniger).
2–3 Wochen Verzug	Launch um 2–3 Wochen schieben; Reihenfolge der Gates nie umdrehen.
Gate nicht bestanden	Kill-Regeln aus Kapitel 6.8: bis zu zwei zusätzliche Zyklen, dann Pivot/Stopp.
Motivationstief	Ein Wochenende ohne Roblox. Dann nur das kleinste nächste Stück bauen und testen.

ANHANG

Anhang A – Der Code des Starter-Kits

Alle Dateien liegen auch im Repo unter `game/src/` (dort sind sie die verbindliche Fassung). In diesem PDF sind Tabulatoren zu 2 Leerzeichen geworden. **Wichtig:** Name, Typ und Ort müssen exakt so angelegt sein (Kapitel 4.2). Der Code wurde mit einem Luau-Parser auf Syntax und mit 71 automatischen Prüfungen auf Server-Verhalten getestet.

A.1 Config

Name	Typ	Ort
Config	ModuleScript	ReplicatedStorage

```
-- Config (ModuleScript) -> ReplicatedStorage
-- Alle Zahlen, Namen und IDs an EINEM Ort. Hier drehst du später am Balancing.

local Config = {}

-- Datenspeicher -----
Config.DATASTORE_NAME = "PlayerData_v1"
Config.AUTOSAVE_SECONDS = 60
Config.LOCK_TIMEOUT_SECONDS = 120 -- so lange gilt eine Sitzungssperre höchstens

-- Standardwerte für neue Spieler (Feld hinzufügen = hier eintragen) -----
Config.DEFAULT_DATA = {
    version = 1,
    coins = 0,
    totalCollected = 0,
    upgradeLevel = 0,
    lastDailyDay = 0,
    dailyStreak = 0,
    onboardingStep = 0,
    purchases = {}, -- zuletzt verarbeitete Kauf-IDs (Schutz vor Doppel-Belohnung)
}

-- Ökonomie (Platzhalter-Loop: Klicken -> Münzen -> Upgrade) -----
Config.COLLECT_BASE = 1 -- Münzen pro Klick auf Level 0
Config.COLLECT_PER_LEVEL = 1 -- zusätzliche Münzen pro Upgrade-Level
Config.UPGRADE_BASE_COST = 10 -- Preis für Level 1
Config.UPGRADE_GROWTH = 1.15 -- Preis wächst pro Level um 15 %
Config.MAX_UPGRADE_LEVEL = 100
Config.MAX_COLLECTS_PER_SECOND = 12 -- Server-Limit gegen Auto-Clicker
Config.DAILY_REWARDS = { 50, 75, 100, 150, 200, 300, 500 } -- 7-Tage-Serie
Config.DAILY_GRACE_DAYS = 1 -- so viele verpasste Tage verzeihen wir

-- Monetarisierung: IDs aus dem Creator Hub eintragen (0 = noch nicht gesetzt) ---
Config.GAME_PASSES = {
    DoubleCoins = 0,
}
```

```

Config.PRODUCTS = {
    Coins500 = { id = 0, coins = 500 },
}

-- Closed Beta: nur eingeladene Tester dürfen rein (BETA_ONLY = true) -----
Config.BETA_ONLY = false
Config.BETA_ALLOWLIST = {
    -- [123456789] = true, -- UserId deines Testers (Profil-URL: roblox.com/users/ZAHL)
}
Config.BETA_GROUP_ID = 0 -- optional: alle Mitglieder dieser Gruppe dürfen rein
Config.BETA_MIN_RANK = 1 -- ab diesem Gruppenrang (1 = jedes Mitglied)

-- Gemeinsame Rechenfunktionen (Client zeigt Preise an, Server prüft sie) -----
function Config.upgradeCost(level: number): number
    return math.floor(Config.UPGRADE_BASE_COST * Config.UPGRADE_GROWTH ^ level)
end

function Config.collectAmount(level: number): number
    return Config.COLLECT_BASE + Config.COLLECT_PER_LEVEL * level
end

function Config.currentDay(): number
    return math.floor(os.time() / 86400) -- Tageszähler in UTC
end

return Config

```

A.2 Main

Name	Typ	Ort
Main	Script	ServerScriptService

```

-- Main (Script) -> ServerScriptService
-- Startet alles in der richtigen Reihenfolge und legt die RemoteEvents an.

local ReplicatedStorage = game:GetService("ReplicatedStorage")

local Services = script.Parent.WaitForChild("Services")
local DataService = require(Services.WaitForChild("DataService"))
local EconomyService = require(Services.WaitForChild("EconomyService"))
local ShopService = require(Services.WaitForChild("ShopService"))
local DailyRewards = require(Services.WaitForChild("DailyRewards"))
local BetaGate = require(Services.WaitForChild("BetaGate"))

-- RemoteEvents anlegen (der Client wartet mit WaitForChild darauf)
local remotes = Instance.new("Folder")
remotes.Name = "Remotes"
for _, name in { "Collect", "BuyUpgrade", "ClaimDaily", "RequestState", "State", "Notice" } do
    local remote = Instance.new("RemoteEvent")
    remote.Name = name
    remote.Parent = remotes
end
remotes.Parent = ReplicatedStorage

-- Verdrahten
DataService.onLoaded(EconomyService.setupPlayer)
EconomyService.setMultiplierProvider(function(player)
    return if ShopService.ownsPass(player, "DoubleCoins") then 2 else 1
end)

BetaGate.start() -- zuerst: Unbefugte fliegen raus, bevor Daten geladen werden
DataService.start()
EconomyService.start(remotes)
DailyRewards.start(remotes)
ShopService.start(function(player)
    EconomyService.pushState(player) -- zeigt den neuen Multiplikator sofort an
end)

print("[Main] Server gestartet.")

```

A.3 DataService

Name	Typ	Ort
DataService	ModuleScript	ServerScriptService → Services

```
-- DataService (ModuleScript) -> ServerScriptService/Services
-- Lädt und speichert Spielerdaten sicher:
-- * Sitzungssperre (verhindert, dass zwei Server gleichzeitig am selben Spieler schreiben)
-- * Wiederholungen bei DataStore-Fehlern, Auto-Save, Speichern beim Verlassen/Shutdown
-- * Studio-Testmodus ohne Speichern, falls der API-Zugriff nicht aktiviert ist

local DataStoreService = game:GetService("DataStoreService")
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local RunService = game:GetService("RunService")

local Config = require(ReplicatedStorage:WaitForChild("Config"))

local DataService = {}

local store = DataStoreService:GetDataStore(Config.DATASTORE_NAME)
local JOB_ID = if game.JobId ~= "" then game.JobId else "studio"

local sessions = {} -- [Player] = Session
local releasing = {} -- [key] = true, solange ein Spieler gerade gespeichert/freigegeben wird
local loadedCallbacks = {}
local started = false

local function deepCopy(value)
    if type(value) ~= "table" then
        return value
    end
    local copy = {}
    for k, v in value do
        copy[k] = deepCopy(v)
    end
    return copy
end

local function reconcile(data, template)
    for key, defaultValue in template do
        if data[key] == nil then
            data[key] = deepCopy(defaultValue)
        end
    end
    return data
end

local function lockedByOther(record)
    local lock = if type(record) == "table" then record.lock else nil
    return lock ~= nil
        and lock.job == JOB_ID
        and os.time() - lock.time < Config.LOCK_TIMEOUT_SECONDS
end

local function withRetries(label, attempts, fn)
    local lastError
    for attempt = 1, attempts do
        local ok, result = pcall(fn)
        if ok then
            return true, result
        end
        lastError = result
        warn(
            string.format(
                "[DataService] %s fehlgeschlagen (Versuch %d/%d): %s",
                label,
                attempt,
                attempts,
                tostring(result)
            )
        )
        if attempt < attempts then
            task.wait(math.min(2 ^ attempt, 8))
        end
    end
    return false, lastError
end
```



```

end

-- Holt die Daten und setzt gleichzeitig unsere Sperre.
local function acquire(key)
    local blocked = false
    local loadedData = nil
    local ok = withRetries("Laden", 3, function()
        store.UpdateAsync(key, function(record)
            blocked = false
            loadedData = nil
            if type(record) ~= "table" then
                record = {}
            end
            if lockedByOther(record) then
                blocked = true
                return nil -- abbrechen, nichts schreiben
            end
            loadedData = record.data
            record.lock = { job = JOB_ID, time = os.time() }
            return record
        end)
    end)
    return ok, blocked, loadedData
end

local function writeSession(session, release)
    local lost = false
    local ok = withRetries("Speichern", 4, function()
        store.UpdateAsync(session.key, function(record)
            lost = false
            if lockedByOther(record) then
                lost = true -- ein anderer Server hat übernommen: NICHT überschreiben
                return nil
            end
        end)
    end)
    return {
        data = session.data,
        lock = if release then nil else { job = JOB_ID, time = os.time() },
    }
end)
end)
if lost then
    warn("[DataService] Sitzungssperre verloren, nicht gespeichert: " .. session.key)
end
return ok and not lost
end

local function saveSession(session, release)
    if session.readOnly then
        return true
    end
    while session.saving do
        task.wait(0.05)
    end
    session.saving = true
    session.dirty = false -- Änderungen während des Speicherns setzen es wieder auf true
    local ok = writeSession(session, release)
    if not ok then
        session.dirty = true
    end
    session.saving = false
    return ok
end

local function endSession(player)
    local session = sessions[player]
    if not session or session.ending then
        return
    end
    session.ending = true
    releasing[session.key] = true
    saveSession(session, true)
    releasing[session.key] = nil
end

```

```

    sessions[player] = nil
end

local function openSession(player, key, loaded, readOnly)
    local data = reconcile(loaded or {}, Config.DEFAULT_DATA)
    data.version = Config.DEFAULT_DATA.version -- hier später Migrationen einbauen
    local session = {
        player = player,
        key = key,
        data = data,
        dirty = true,
        saving = false,
        ending = false,
        readOnly = readOnly == true,
    }
    sessions[player] = session
    for _, callback in loadedCallbacks do
        task.spawn(callback, player, data)
    end
end

local function onPlayerAdded(player)
    local key = "Player_" .. player.UserId
    while releasing[key] do
        task.wait(0.1) -- alte Sitzung dieses Spielers wird noch gespeichert
    end

    local ok, blocked, loaded
    for _ = 1, 5 do
        ok, blocked, loaded = acquire(key)
        if not ok or not blocked then
            break
        end
        task.wait(3) -- anderer Server speichert noch, kurz warten
        if player.Parent ~= Players then
            return
        end
    end

    if player.Parent ~= Players then
        if ok and not blocked then -- Sperre, die wir gerade gesetzt haben, wieder lösen
            writeSession({ key = key, data = reconcile(loaded or {}, Config.DEFAULT_DATA) }, true)
        end
        return
    end

    if ok and not blocked then
        openSession(player, key, loaded, false)
    elseif RunService:IsStudio() then
        warn(
            "[DataService] Kein DataStore-Zugriff. Testmodus OHNE Speichern. "
            .. "(Game Settings > Security > Enable Studio Access to API Services)"
        )
        openSession(player, key, nil, true)
    elseif blocked then
        player:Kick(
            "Deine Daten werden noch gespeichert. Bitte versuche es in einer Minute erneut."
        )
    else
        player:Kick("Deine Daten konnten nicht geladen werden. Bitte versuche es später erneut.")
    end
end

local function autosaveLoop()
    while true do
        task.wait(Config.AUTOSAVE_SECONDS)
        local pending = {}
        for _, session in sessions do
            if session.dirty and not session.ending then
                table.insert(pending, session)
            end
        end
    end
end

```

```

        for _, session in pending do
            task.spawn(saveSession, session, false)
            task.wait(0.2) -- Anfragen entzerren
        end
    end
end

-- Öffentliche API -----

function DataService.start()
    if started then
        return
    end
    started = true
    Players.PlayerAdded:Connect(onPlayerAdded)
    for _, player in Players:GetPlayers() do
        task.spawn(onPlayerAdded, player)
    end
    Players.PlayerRemoving:Connect(endSession)
    task.spawn(autosaveLoop)
    game:BindToClose(function()
        for player in sessions do
            task.spawn(endSession, player)
        end
        while next(sessions) ~= nil or next(releasing) ~= nil do
            task.wait(0.1)
        end
    end)
end

function DataService.onLoaded(callback)
    table.insert(loadedCallbacks, callback)
end

function DataService.get(player)
    local session = sessions[player]

    return if session then session.data else nil
end

function DataService.isLoaded(player)
    return sessions[player] ~= nil
end

function DataService.markDirty(player)
    local session = sessions[player]
    if session then
        session.dirty = true
    end
end

function DataService.saveNow(player)
    local session = sessions[player]
    if not session then
        return false
    end
    return saveSession(session, false)
end

return DataService

```

A.4 Analytics

Name	Typ	Ort
Analytics	ModuleScript	ServerScriptService → Services

```

-- Analytics (ModuleScript) -> ServerScriptService/Services
-- Dünne Hülle um Roblox' AnalyticsService. Jeder Aufruf steckt in pcall:
-- Ändert Roblox einen Funktionsnamen, läuft dein Spiel trotzdem weiter
-- (du siehst dann EINE Warnung im Output). Signaturen in der aktuellen Doku prüfen!

local AnalyticsService = game:GetService("AnalyticsService")

local Analytics = {}

local warned = false

local function safe(fn)
    local ok, err = pcall(fn)
    if not ok and not warned then
        warned = true
        warn("[Analytics] Aufruf fehlgeschlagen (Signatur in der Doku prüfen): " .. tostring(err))
    end
end

-- Onboarding-Trichter: step 1, 2, 3 ... in der Reihenfolge, in der Spieler sie erreichen
function Analytics.step(player, step: number, name: string)
    safe(function()
        AnalyticsService:LogOnboardingFunnelStepEvent(player, step, name)
    end)
end

-- Eigene Ereignisse, z. B. Analytics.custom(player, "daily_claimed", 1)
function Analytics.custom(player, eventName: string, value: number?)
    safe(function()
        AnalyticsService:LogCustomEvent(player, eventName, value or 1)
    end)
end

-- Wirtschaft: isSource = true (Münzen kommen rein), false (Münzen werden ausgegeben).
-- reason ist ein kurzer Text wie "upgrade", "daily" oder "purchase:Coins500" (landet als itemSku).
function Analytics.economy(
    player,
    isSource: boolean,
    currency: string,
    endingBalance: number,
    amount: number,
    reason: string
)
    safe(function()
        local flow = if isSource
            then Enum.AnalyticsEconomyFlowType.Source
            else Enum.AnalyticsEconomyFlowType.Sink
        local txType = if string.sub(reason, 1, 8) == "purchase" then "IAP" else "Gameplay"
        pcall(function() -- bevorzugt die offiziellen Enum-Namen, falls vorhanden
            txType = Enum.AnalyticsEconomyTransactionType[txType].Name
        end)
        AnalyticsService:LogEconomyEvent(
            player,
            flow,
            currency,
            endingBalance,
            amount,
            txType,
            reason
        )
    end)
end

return Analytics

```

A.5 EconomyService

Name	Typ	Ort
EconomyService	ModuleScript	ServerScriptService → Services

```
-- EconomyService (ModuleScript) -> ServerScriptService/Services
-- Die Wahrheit über Münzen und Upgrades liegt IMMER hier auf dem Server.
-- Der Client darf nur bitten ("ich habe geklickt"), nie sagen "gib mir 1000 Münzen".

local Players = game.GetService("Players")
local ReplicatedStorage = game.GetService("ReplicatedStorage")

local Config = require(ReplicatedStorage:WaitForChild("Config"))
local DataService = require(script.Parent.DataService)
local Analytics = require(script.Parent.Analytics)

local EconomyService = {}

local remotes = nil
local multiplierProvider = function(_player)
    return 1
end
local buckets = {} -- [Player] = { [aktion] = { tokens, last } }

-- Rate-Limit (Token-Bucket): erlaubt `perSecond` Aktionen pro Sekunde, Burst bis `burst`
local function allow(player, action: string, perSecond: number, burst: number): boolean
    local playerBuckets = buckets[player]
    if not playerBuckets then
        playerBuckets = {}
        buckets[player] = playerBuckets
    end
    local now = os.clock()
    local bucket = playerBuckets[action]
    if not bucket then
        bucket = { tokens = burst, last = now }
        playerBuckets[action] = bucket
    end
    bucket.tokens = math.min(burst, bucket.tokens + (now - bucket.last) * perSecond)
    bucket.last = now
    if bucket.tokens >= 1 then
        bucket.tokens -= 1
    end

    return true
end
return false
end

local function reachStep(player, data, step: number, name: string)
    if data.onboardingStep < step then
        data.onboardingStep = step
        Analytics.step(player, step, name)
    end
end

local function notice(player, text: string)
    if remotes then
        remotes.Notice:FireClient(player, text)
    end
end

function EconomyService.setMultiplierProvider(fn)
    multiplierProvider = fn
end

function EconomyService.getState(player)
    local data = DataService.get(player)
    if not data then
        return nil
    end
    local multiplier = multiplierProvider(player)
    return {
        coins = data.coins,
        level = data.upgradeLevel,
        perClick = Config.collectAmount(data.upgradeLevel) * multiplier,
        nextCost = if data.upgradeLevel >= Config.MAX_UPGRADE_LEVEL
            then nil
            else Config.upgradeCost(data.upgradeLevel),
        multiplier = multiplier,
    }
end
```

```

        dailyReady = data.lastDailyDay ~= Config.currentDay(),
        streak = data.dailyStreak,
    }
end

function EconomyService.pushState(player)
    local state = EconomyService.getState(player)
    if not state or not remotes then
        return
    end
    local stats = player:FindFirstChild("leaderstats")
    local coinsValue = if stats then stats:FindFirstChild("Coins") else nil
    if coinsValue then
        coinsValue.Value = math.floor(state.coins)
    end
    remotes.State:FireClient(player, state)
end

-- Nur für Server-Code (Belohnungen, Käufe). Gibt true zurück, wenn gutgeschrieben.
function EconomyService.addCoins(player, amount: number, reason: string): boolean
    local data = DataService.get(player)
    if not data or type(amount) ~= "number" or amount ~= amount or amount <= 0 then
        return false
    end
    data.coins += amount
    data.totalCollected += amount
    DataService.markDirty(player)
    Analytics.economy(player, true, "Coins", data.coins, amount, reason)
    EconomyService.pushState(player)
    return true
end

function EconomyService.setupPlayer(player, _data)
    local stats = Instance.new("Folder")
    stats.Name = "leaderstats"
    local coins = Instance.new("IntValue")

    coins.Name = "Coins"
    coins.Value = 0
    coins.Parent = stats
    stats.Parent = player
    local data = DataService.get(player)
    if data then
        reachStep(player, data, 1, "Joined")
    end
    EconomyService.pushState(player)
end

local function onCollect(player)
    local data = DataService.get(player)
    if not data then
        return
    end
    if
        not allow(player, "Collect", Config.MAX_COLLECTS_PER_SECOND, Config.MAX_COLLECTS_PER_SECOND)
    then
        return
    end
    local amount = Config.collectAmount(data.upgradeLevel) * multiplierProvider(player)
    data.coins += amount
    data.totalCollected += amount
    DataService.markDirty(player)
    reachStep(player, data, 2, "FirstCollect")
    EconomyService.pushState(player)
end

local function onBuyUpgrade(player)
    local data = DataService.get(player)
    if not data then
        return
    end
    if not allow(player, "BuyUpgrade", 4, 4) then
        return
    end

```

```

end
if data.upgradeLevel >= Config.MAX_UPGRADE_LEVEL then
    notice(player, "Maximales Level erreicht!")
    return
end
local cost = Config.upgradeCost(data.upgradeLevel)
if data.coins < cost then
    notice(player, "Nicht genug Münzen!")
    return
end
data.coins -= cost
data.upgradeLevel += 1
DataService.markDirty(player)
Analytics.economy(player, false, "Coins", data.coins, cost, "upgrade")
reachStep(player, data, 3, "FirstUpgrade")
EconomyService.pushState(player)
end

function EconomyService.start(remoteFolder)
    remotes = remoteFolder
    remotes.Collect.OnServerEvent:Connect(onCollect)
    remotes.BuyUpgrade.OnServerEvent:Connect(onBuyUpgrade)
    remotes.RequestState.OnServerEvent:Connect(function(player)
        if allow(player, "RequestState", 2, 3) then
            EconomyService.pushState(player)
        end
    end)
    Players.PlayerRemoving:Connect(function(player)
        buckets[player] = nil
    end)
end

EconomyService.reachStep = reachStep

return EconomyService

```

A.6 DailyRewards

Name	Typ	Ort
DailyRewards	ModuleScript	ServerScriptService → Services

```

-- DailyRewards (ModuleScript) -> ServerScriptService/Services
-- Tägliche Belohnung mit Serie (Streak). Ein verpasster Tag wird verziehen (Config.DAILY_GRACE_DAYS).
-- Tageswechsel = 00:00 UTC (in Deutschland 01:00 bzw. 02:00 Uhr).

local ReplicatedStorage = game:GetService("ReplicatedStorage")

local Config = require(ReplicatedStorage:WaitForChild("Config"))
local DataService = require(script.Parent.DataService)
local EconomyService = require(script.Parent.EconomyService)
local Analytics = require(script.Parent.Analytics)

local DailyRewards = {}

local remotes = nil
local claiming = {} -- verhindert Doppelklicks, während gespeichert wird

-- Gibt (ok, belohnung, serie) zurück
function DailyRewards.claim(player)
    local data = DataService.get(player)
    if not data then
        return false
    end
    local today = Config.currentDay()
    if data.lastDailyDay == today then
        return false
    end
    local gap = today - data.lastDailyDay
    local streak
    if data.lastDailyDay ~= 0 and gap <= 1 + Config.DAILY_GRACE_DAYS then
        streak = data.dailyStreak + 1
    else
        streak = 1
    end
    local rewards = Config.DAILY_REWARDS
    local reward = rewards[(streak - 1) % #rewards + 1]

```

```

data.lastDailyDay = today -- zuerst eintragen, dann belohnen (kein Doppel-Claim möglich)
data.dailyStreak = streak
EconomyService.addCoins(player, reward, "daily")
EconomyService.reachStep(player, data, 4, "FirstDaily")
Analytics.custom(player, "daily_claimed", streak)
return true, reward, streak
end

function DailyRewards.start(remoteFolder)
    remotes = remoteFolder
    remotes.ClaimDaily.OnServerEvent:Connect(function(player)
        if claiming[player] then
            return
        end
        claiming[player] = true
        local ok, reward, streak = DailyRewards.claim(player)
        if ok then
            remotes.Notice:FireClient(player, string.format("Tag %d: +%d Münzen!", streak, reward))
            DataService.saveNow(player)
        else
            remotes.Notice:FireClient(player, "Heute schon abgeholt. Komm morgen wieder!")
        end
        claiming[player] = nil
    end)
end

return DailyRewards

```

A.7 ShopService

Name	Typ	Ort
ShopService	ModuleScript	ServerScriptService → Services

```

-- ShopService (ModuleScript) -> ServerScriptService/Services
-- Game Passes (dauerhafte Käufe) und Developer Products (Käufe, die man wiederholen kann).
-- IDs trägst du in Config.GAME_PASSES / Config.PRODUCTS ein (0 = noch nicht gesetzt).

local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")

local Config = require(ReplicatedStorage:WaitForChild("Config"))
local DataService = require(script.Parent.DataService)
local EconomyService = require(script.Parent.EconomyService)

local ShopService = {}

local MAX_PURCHASE_HISTORY = 100
local owned = {} -- [Player] = { [PassName] = true }
local onChanged = function(_player) end

function ShopService.ownsPass(player, passName: string): boolean
    local passes = owned[player]
    return passes ~= nil and passes[passName] == true
end

local function loadOwnership(player)
    local passes = {}
    owned[player] = passes
    for name, id in Config.GAME_PASSES do
        if id ~= 0 then
            local ok, result = pcall(function()
                return MarketplaceService.UserOwnsGamePassAsync(player.UserId, id)
            end)
            if ok and result == true then
                passes[name] = true
            end
        end
    end
end
end

```



```

    onChanged(player)
end

local function alreadyProcessed(data, purchaseId: string): boolean
    for _, id in data.purchases do
        if id == purchaseId then
            return true
        end
    end
    return false
end

-- Roblox ruft das für jeden Developer-Product-Kauf auf und wiederholt es, bis wir
-- PurchaseGranted zurückgeben. Darum: idempotent (doppelt aufgerufen = nur einmal belohnen).
function ShopService.processReceipt(receiptInfo)
    local notYet = Enum.ProductPurchaseDecision.NotProcessedYet
    local granted = Enum.ProductPurchaseDecision.PurchaseGranted

    local player = Players:GetPlayerByUserId(receiptInfo.PlayerId)
    if not player then
        return notYet -- Spieler ist weg; Roblox probiert es beim nächsten Beitritt erneut
    end
    local data = DataService.get(player)
    if not data then
        return notYet
    end

    local purchaseId = tostring(receiptInfo.PurchaseId)
    if alreadyProcessed(data, purchaseId) then
        return if DataService.saveNow(player) then granted else notYet
    end

    for name, product in Config.PRODUCTS do
        if product.id ~= 0 and product.id == receiptInfo.ProductId then
            EconomyService.addCoins(player, product.coins, "purchase:" .. name)
            table.insert(data.purchases, purchaseId)

            while #data.purchases > MAX_PURCHASE_HISTORY do
                table.remove(data.purchases, 1)
            end
            return if DataService.saveNow(player) then granted else notYet
        end
    end

    warn("[ShopService] Unbekanntes Produkt: " .. tostring(receiptInfo.ProductId))
    return notYet
end

function ShopService.start(onOwnershipChanged)
    onChanged = onOwnershipChanged or onChanged

    Players.PlayerAdded:Connect(function(player)
        task.spawn(loadOwnership, player)
    end)
    for _, player in Players:GetPlayers() do
        task.spawn(loadOwnership, player)
    end
    Players.PlayerRemoving:Connect(function(player)
        owned[player] = nil
    end)

    MarketplaceService.PromptGamePassPurchaseFinished:Connect(function(player, passId, wasPurchased)
        if not wasPurchased then
            return
        end
        for name, id in Config.GAME_PASSES do
            if id ~= 0 and id == passId then
                owned[player] = owned[player] or {}
                owned[player][name] = true
                onChanged(player)
            end
        end
    end)
end

MarketplaceService.ProcessReceipt = ShopService.processReceipt
end

return ShopService

```

A.8 BetaGate

Name	Typ	Ort
BetaGate	ModuleScript	ServerScriptService → Services

```

-- BetaGate (ModuleScript) -> ServerScriptService/Services
-- Lässt in der Closed Beta nur eingeladene Tester spielen. Aus für den Launch: Config.BETA_ONLY = false.

local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")

local Config = require(ReplicatedStorage:WaitForChild("Config"))

local BetaGate = {}

local function isAllowed(player): boolean
    if Config.BETA_ALLOWLIST[player.UserId] then
        return true
    end
    if Config.BETA_GROUP_ID ~= 0 then
        local ok, rank = pcall(function()
            return player:GetRankInGroup(Config.BETA_GROUP_ID)
        end)
        if ok and rank >= Config.BETA_MIN_RANK then
            return true
        end
    end
    return false
end

local function check(player)
    if not isAllowed(player) then
        player:Kick("Diese Beta ist nur für eingeladene Tester. Bis zum Launch!")
    end
end

function BetaGate.start()
    if not Config.BETA_ONLY then
        return
    end
    Players.PlayerAdded:Connect(check)

    for _, player in Players:GetPlayers() do
        task.spawn(check, player)
    end
end

return BetaGate

```

A.9 Client

Name	Typ	Ort
Client	LocalScript	StarterPlayer → StarterPlayerScripts

```
-- Client (LocalScript) -> StarterPlayer/StarterPlayerScripts
-- Baut eine einfache, handytaugliche Oberfläche per Code und zeigt den Zustand an,
-- den der Server schickt. Ersetze das später durch dein eigenes UI-Design.

local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")

local player = Players.LocalPlayer
local Config = require(ReplicatedStorage.WaitForChild("Config"))
local remotes = ReplicatedStorage.WaitForChild("Remotes")

local function formatNumber(n: number): string
    local suffixes = { "", "K", "M", "B", "T" }
    local index = 1
    while n >= 1000 and index < #suffixes do
        n /= 1000
        index += 1
    end
    if index == 1 then
        return tostring(math.floor(n))
    end
    return string.format("%.1f%s", n, suffixes[index])
end

local gui = Instance.new("ScreenGui")
gui.Name = "MainUI"
gui.ResetOnSpawn = false
gui.ZIndexBehavior = Enum.ZIndexBehavior.Sibling
gui.Parent = player.WaitForChild("PlayerGui")

local function styleText(label, maxSize: number)
    label.Font = Enum.Font.GothamBold
    label.TextScaled = true
    label.TextColor3 = Color3.new(1, 1, 1)
    local limit = Instance.new("UITextSizeConstraint")

    limit.MaxTextSize = maxSize
    limit.Parent = label
end

local function makeLabel(name: string, position: UDim2, size: UDim2)
    local label = Instance.new("TextLabel")
    label.Name = name
    label.AnchorPoint = Vector2.new(0.5, 0.5)
    label.Position = position
    label.Size = size
    label.BackgroundTransparency = 1
    label.Parent = gui
    styleText(label, 30)
    return label
end

local function makeButton(name: string, text: string, position: UDim2, size: UDim2, color: Color3)
    local button = Instance.new("TextButton")
    button.Name = name
    button.Text = text
    button.AnchorPoint = Vector2.new(0.5, 0.5)
    button.Position = position
    button.Size = size
    button.BackgroundColor3 = color
    button.Parent = gui
    styleText(button, 26)
    local corner = Instance.new("UICorner")
    corner.CornerRadius = UDim.new(0, 12)
    corner.Parent = button
    local sizeLimit = Instance.new("UISizeConstraint")
    sizeLimit.MinSize = Vector2.new(110, 46) -- Mindestgröße zum Antippen
    sizeLimit.MaxSize = Vector2.new(320, 90)
    sizeLimit.Parent = button
    return button
end
```

```

local coinsLabel = makeLabel("Coins", UDim2.fromScale(0.5, 0.08), UDim2.fromScale(0.4, 0.07))
local hintLabel = makeLabel("Hint", UDim2.fromScale(0.5, 0.68), UDim2.fromScale(0.6, 0.06))
local noticeLabel = makeLabel("Notice", UDim2.fromScale(0.5, 0.3), UDim2.fromScale(0.7, 0.06))
hintLabel.Text = "Tippe auf SAMMELN!"
noticeLabel.Text = ""

local collectButton = makeButton(
    "Collect",
    "SAMMELN",
    UDim2.fromScale(0.5, 0.82),
    UDim2.fromScale(0.34, 0.14),
    Color3.fromRGB(37, 99, 235)
)
local upgradeButton = makeButton(
    "Upgrade",
    "UPGRADE",
    UDim2.fromScale(0.2, 0.82),
    UDim2.fromScale(0.26, 0.11),
    Color3.fromRGB(22, 163, 74)
)
local dailyButton = makeButton(
    "Daily",
    "TÄGLICH",
    UDim2.fromScale(0.8, 0.82),
    UDim2.fromScale(0.26, 0.11),
    Color3.fromRGB(245, 158, 11)
)
local shopButton = makeButton(
    "Shop",
    "2x MÜNZEN",
    UDim2.fromScale(0.5, 0.18),
    UDim2.fromScale(0.3, 0.08),
    Color3.fromRGB(147, 51, 234)
)

local noticeToken = 0

```

```

local function showNotice(text: string)
    noticeToken += 1
    local myToken = noticeToken
    noticeLabel.Text = text
    task.delay(2.5, function()
        if noticeToken == myToken then
            noticeLabel.Text = ""
        end
    end)
end

local function applyState(state)
    coinsLabel.Text = "Münzen: " .. formatNumber(state.coins)
    collectButton.Text = "SAMMELN +" .. formatNumber(state.perClick)
    if state.nextCost then
        upgradeButton.Text = "UPGRADE " .. formatNumber(state.nextCost)
        upgradeButton.BackgroundColor3 = if state.coins >= state.nextCost
            then Color3.fromRGB(22, 163, 74)
            else Color3.fromRGB(100, 116, 139)
    else
        upgradeButton.Text = "MAX"
    end
    dailyButton.Text = if state.dailyReady then "TÄGLICH" else "MORGEN"
    dailyButton.BackgroundColor3 = if state.dailyReady
        then Color3.fromRGB(245, 158, 11)
        else Color3.fromRGB(100, 116, 139)
    shopButton.Visible = state.multiplier == 1
    hintLabel.Visible = state.coins == 0 and state.level == 0
end

collectButton.Activated:Connect(function()
    remotes.Collect:FireServer()
end)
upgradeButton.Activated:Connect(function()
    remotes.BuyUpgrade:FireServer()
end)

```

```
dailyButton.Activated:Connect(function()
    remotes.ClaimDaily:FireServer()
end)
shopButton.Activated:Connect(function()
    local id = Config.GAME_PASSES.DoubleCoins
    if id ~= 0 then
        MarketplaceService.PromptGamePassPurchase(player, id)
    else
        showNotice("Trage zuerst die Game-Pass-ID in Config ein.")
    end
end)

remotes.State.OnClientEvent:Connect(applyState)
remotes.Notice.OnClientEvent:Connect(showNotice)
remotes.RequestState:FireServer() -- Startzustand anfordern (falls wir den ersten Push verpasst haben)
```

ANHANG

Anhang B – Checklisten zum Abhaken

B.1 Pre-Launch-Checkliste

Technik

- ☐ Alle Services im Output ohne rote Zeilen (F5 + Mehrspieler)
- ☐ 100× Join/Leave ohne Datenverlust
- ☐ DataStore-Ausfall simuliert: keine leeren Spielstände
- ☐ Auf Low-End-Handy getestet, keine Abstürze
- ☐ Anti-Exploit-Selbsttest bestanden (Remotes mit Unsinn)
- ☐ Server-Neustart/Shutdown speichert alle Spieler
- ☐ Wiederherstellung eines Spielstands einmal geübt

Wirtschaft & Shop

- ☐ Alle Pässe/Produkte angelegt, IDs in Config, Preise geprüft
- ☐ Testkauf: Gutschrift korrekt, kein Doppelkauf
- ☐ Keine bezahlten Zufallsitems (oder Quoten offengelegt)
- ☐ Ökonomie-Tabelle geprüft, kein Spieler wartet > 90 s ohne Fortschritt
- ☐ Shop-Pop-ups erst nach 3+ Minuten

Inhalt & Spielgefühl

- ☐ Erste Belohnung < 60 s (Stoppuhr, Neuling-Konto)
- ☐ Inhalt für ≥ 2–3 Stunden
- ☐ Täglich/Streak funktioniert über Tageswechsel
- ☐ Texte DE + EN, von Muttersprachler gegengelesen

Store-Seite

- ☐ Icon, 3–5 Thumbnails (Gewinner aus A/B-Test)
- ☐ Titel, Beschreibung, Genre, Codes
- ☐ Alters-/Maturity-Fragebogen ehrlich ausgefüllt
- ☐ Handy/Tablet/PC aktiviert

Marketing & Community

- ☐ 15–20 Kurzvideos auf Vorrat
- ☐ 30 Creator angeschrieben, Codes vergeben
- ☐ Discord (13+) mit Regeln und 2 Moderatoren
- ☐ Launch-Beitrag und Patchnotes-Vorlage fertig

Recht & Geld

- ☐ Steuerliche Erfassung geklärt (Steuerberater)
- ☐ W-8BEN/Taxes-Seite hinterlegt, Auszahlung getestet/verstanden
- ☐ Impressum + Datenschutz für Website/Social (wenn nötig)
- ☐ Alle Assets/Musik lizenziert, keine Marken im Namen

☐ Prüfliste F1–F9 (Kapitel 9.8) abgehakt

Betrieb

- ☐ Launch-Runbook geschrieben, von zweiter Person gelesen
- ☐ Schichtplan 72 h
- ☐ Hotfix-/Rollback-Weg geübt
- ☐ Notfallschalter (Config) getestet
- ☐ BetaGate: BETA_ONLY = false für den Launch (nicht vergessen!)

B.2 Launch-Tag und die ersten 7 Tage

- ☐ Beta-Schalter AUS und veröffentlicht
- ☐ Spiel öffentlich, Store-Seite geprüft (Bilder, Titel, Links)
- ☐ Creator-Posts + Ankündigung **gleichzeitig** freigegeben
- ☐ Kleines Werbebudget aktiviert (Limit pro Tag!)
- ☐ Dashboard offen: CCU, Fehler, Käufe
- ☐ Server-Log (F9) in den ersten 2 Stunden alle 15 Minuten prüfen
- ☐ Discord: Support-Schicht besetzt
- ☐ Erste Käufe prüfen (kam die Gutschrift an?)
- ☐ Nach 6 h: Zwischenstand notieren
- ☐ Nach 24 h: Hotfix-Fenster, Top-3-Probleme
- ☐ Nach 72 h: Review (stabil? Retention?)
- ☐ Nach 7 Tagen: Update #1 ankündigen

B.3 Wöchentliches KPI-Blatt (kopieren)

Kennzahl	Woche 1	Woche 2	Woche 3	Woche 4	Ziel
DAU (Ø)					steigt
D1-Retention					≥ 30 %
D7-Retention					≥ 12 %
D28-Retention					steigt
Ø Session (Min)					≥ 15
Zahler-Quote					2–3 %
ARPPDAU (\$)					messen
Fehler-/Absturzrate					< 0,5 %
Größtes Problem					
Hypothese dieser Woche					
Ergebnis der letzten Hypothese					

B.4 Monatliche Checks

- ☐ Creator Hub, DevForum und Newsroom auf Regeländerungen durchsehen (Auszahlung, Rewards, Altersregeln)

- ☐ Backup-/Restore-Test durchführen
- ☐ Anti-Exploit-Log auswerten, Konten prüfen
- ☐ Einnahmen gegen Kosten abgleichen; Steuer-Rücklage aufstocken
- ☐ Event für nächsten Monat in 6-Wochen-Planung prüfen
- ☐ Community-Umfrage auswerten
- ☐ Eine Sache lernen/ausprobieren, die du noch nicht kannst

ANHANG

Anhang C – Glossar

Begriff	Bedeutung
A/B-Test	Zwei Varianten gleichzeitig an verschiedene Spieler ausspielen, um zu sehen, welche besser wirkt.
ARPDau	Umsatz pro aktivem Spieler und Tag (Average Revenue Per Daily Active User).
CCU	Concurrent Users: Spieler, die gleichzeitig online sind.
Client	Das Gerät des Spielers (Handy/PC). Darf nur bitten, nie entscheiden.
Config	Eine Datei, in der alle Zahlen und IDs deines Spiels zentral stehen.
Creator Hub	Roblox-Webseite (create.roblox.com) für Verwaltung, Analytics, Monetarisierung.
Creator Rewards	Programm, das Entwickler anhand von Engagement (u. a. Spielzeit) belohnt Details prüfen .
DAU	Daily Active Users: aktive Spieler pro Tag.
DataStore	Roblox-Cloud-Speicher für Spielerdaten.
DevEx	Developer Exchange: Umtausch verdienter Robux in echtes Geld.
Developer Product	Kaufbarer Artikel, der beliebig oft gekauft werden kann.
Discovery	Roblox' System, das Spielern Spiele vorschlägt (Empfohlen für dich).
D1 / D7 / D28	Anteil der Spieler, die nach 1 / 7 / 28 Tagen wiederkommen (Retention).
Experience	Roblox-Wort für «Spiel».
Exploit	Schummelsoftware oder ein Fehler, der unerlaubte Vorteile erlaubt.
Explorer	Studio-Fenster mit dem Objektbaum deines Spiels.
FTUE	First-Time User Experience: die ersten Minuten eines Neulings.
Game Pass	Einmalig kaufbarer, dauerhafter Vorteil.
Gate	Entscheidungspunkt am Ende einer Phase: Weiter, Nachbessern oder Stopp.
Graybox	Spielbare Rohversion mit Platzhalter-Grafik.
Hard Launch / Soft Launch	Öffentlicher Start mit Marketing / öffentlich ohne Marketing zum Stabilisieren.
Hotfix	Schnelle Fehlerbehebung außerhalb des normalen Update-Plans.
Idempotent	Mehrfaches Ausführen hat nur einmal Wirkung (wichtig bei Käufen).
KPI	Schlüsselkennzahl (z. B. D1, Session, Zahler-Quote).
Live-Ops	Laufender Betrieb: Updates, Events, Community, Messen.
LocalScript	Skript, das beim Spieler läuft (Oberfläche, Eingaben).
Luau	Roblox' Programmiersprache.
Maturity-Fragebogen	Pflicht-Altersfreigabe-Fragen im Creator Hub.
ModuleScript	Wiederverwendbare Skript-Bibliothek, die mit <code>require</code> geladen wird.
Output	Studio-Fenster mit Meldungen und Fehlern deines Codes.

Begriff	Bedeutung
Premium	Roblox-Abo; Entwickler werden für Spielzeit von Premium-Nutzern vergütet △ .
ProcessReceipt	Server-Funktion, die Käufe von Developer Products bestätigt.
Rate-Limit	Begrenzung, wie oft eine Aktion pro Sekunde erlaubt ist (gegen Spam/Cheats).
RemoteEvent	Briefkasten für Nachrichten zwischen Client und Server.
Retention	Wiederkehr-Quote: Wie viele Spieler kommen zurück.
Robux	Roblox-Währung.
Rojo	Werkzeug, das Code-Dateien ins Studio synchronisiert.
Script	Skript, das auf dem Server läuft.
Server	Roblox-Rechner, auf dem das Spiel läuft. Entscheidet über alles Wichtige.
Service	Roblox-Baustein (z. B. Players, DataStoreService) oder ein Modul deines Spiels.
Sitzungssperre	Markierung im DataStore, dass ein Server gerade den Spieler bearbeitet.
Thumbnail	Das große Vorschaubild deines Spiels im Store.
Tipalti	Zahlungsdienst, über den DevEx ausgezahlt wird.
UserId	Eindeutige Zahl eines Roblox-Kontos.
Zahler-Quote	Anteil der Spieler, die mindestens einmal kaufen.

ANHANG

Anhang D – Hilfe holen

Du musst nie allein weiterkommen. Je genauer deine Meldung, desto schneller die Lösung.

D.1 So meldest du ein Problem

1. Schreibe die **Schritt-Nummer** aus diesem Handbuch.
2. Kopiere die **komplette rote Zeile** aus dem Output (nicht abtippen).
3. Mache einen **Screenshot vom Explorer** (aufgeklappt) und, wenn es um Oberfläche geht, vom Spiel.
4. Beschreibe: **Was habe ich erwartet? Was ist passiert?**
5. Sage dazu, ob es im **Studio** oder im **veröffentlichten Spiel** passiert.

TIPP

Echte Screenshots kann ich dir nicht liefern, aber **deine** kann ich lesen. Schick sie mir einfach im Chat: Dann zeige ich dir die genaue Stelle und passe das Handbuch für dich an.

D.2 Prompt-Bibliothek (kopieren, Klammern ausfüllen)

Fehler im Output

Ich arbeite an Schritt [Nummer] im Handbuch. Im Output steht folgende rote Zeile: [KOMPLETTE ZEILE EINFÜGEN]. Hier ist ein Screenshot meines Explorers: [BILD]. Was ist falsch und was soll ich genau klicken?

Mein Loop

Mein Spiel in 5 Sätzen: [AKTION], [BELOHNUNG], [ERSTES UPGRADE], [ERSTES ZIEL], [SOZIALES ELEMENT]. Baue mir daraus nach dem 7-Schritte-Rezept (Kapitel 5.1) Server-Code, Client-Code, Config-Werte und einen Testplan.

Ökonomie

Meine Spielschleife: [BESCHREIBUNG]. Schlage mir Quellen, Senken, Preise und eine Meilenstein-Tabelle für die ersten 60 Minuten vor, und erkläre mir, wie ich sie in Google Sheets nachrechne.

Beta-Zahlen

Meine Beta-Zahlen: D1 [...] %, D7 [...] %, Session [...] Min, Zahler-Quote [...] %, größte Absprungstelle im Trichter: [...]. Was ist das größte Problem und welche EINE Änderung teste ich als Erstes (mit Hypothese und Messung)?

Store-Seite

Mein Spiel: [1-SATZ-LOOP], Zielgruppe [...]. Schreibe mir 5 Titelvorschläge (DE/EN), 3 Beschreibungstexte und 5 Thumbnail-Ideen mit Bildaufbau.

Creator-Pitch

Hier ist das Video eines Creators: [LINK/BESCHREIBUNG]. Schreibe mir eine kurze, persönliche Anfrage (DE + EN) mit Early-Access und Code.

Update planen

Meine letzten Zahlen: [...]. Plane mir das Wochen-Update (Mo–Fr) und das nächste Monats-Event: Inhalt, Aufwand, Messgröße, Teaser-Clips.

Wochen-Review

Hier sind meine KPI der letzten 4 Wochen: [TABELLE]. Zeige mir Trends, Auffälligkeiten und die 3 wichtigsten Entscheidungen für nächste Woche.

D.3 Typische Probleme

Symptom	Ursache	Lösung
Spiel startet nicht / weißer Bildschirm	Output prüfen; oft fehlt ein Objekt oder ein Name ist falsch.	Kapitel 4.3, Tabelle.
«Infinite yield possible ...»	WaitForChild wartet ewig auf ein Objekt, das es nicht gibt.	Name/Ort im Explorer vergleichen.
«Script timeout: exhausted allowed execution time»	Eine Schleife läuft ohne Pause.	<code>task.wait()</code> in Schleifen einbauen.
Spielstand speichert nicht (nur im Studio)	API-Zugriff aus.	Game Settings → Security (Kapitel 3.4), Studio neu starten.
«Request was throttled / 429» in Warnungen	Zu viele DataStore-Anfragen.	Autosave-Intervall nicht verkürzen; nur speichern, wenn etwas geändert wurde (macht das Kit).
Kaufdialog öffnet nicht	ID ist 0 oder Pass nicht «zum Verkauf».	ID in Config prüfen, Pass-Seite prüfen, veröffentlichtes Spiel testen.
Käufe kommen doppelt an	Belohnung im Client oder ohne Kauf-ID-Prüfung.	Nur im ProcessReceipt belohnen (macht das Kit).
Spieler wird mit «Daten werden noch gespeichert» getrennt	Sitzungssperre eines anderen Servers.	Nach 1–2 Minuten erneut beitreten; Sperren laufen nach 120 s ab.
Alles ruckelt auf dem Handy	Zu viele Parts/Partikel/Skripte pro Frame.	Kapitel 5.8; MicroProfiler (Strg+F6).
Studio stürzt ab	Speicher/Plugin.	Plugins deaktivieren, Autosave-Wiederherstellung beim Neustart nutzen.
Spiel wird moderiert/entfernt	Verstoß gegen Richtlinien oder falsche Einstufung.	Hinweis im Creator Hub lesen, Fragebogen/Inhalte anpassen, Einspruch einlegen.

ANHANG

Anhang E – Quellen und Stand

Recherchestand: 9. Oktober 2026. Die Seiten von `create.roblox.com` waren aus meiner Umgebung nicht erreichbar (Netzwerkrichtlinie); die dortigen Aussagen stammen aus Suchzusammenfassungen. Sekundärquellen (Blogs, Presse) können sich irren. **Entscheidend ist immer, was im Creator Hub steht.**

- Roblox Newsroom: Optimizing Discovery (Juni 2026)
<https://about.roblox.com/newsroom/2026/06/optimizing-discovery-great-games-reach-millions-players-roblox>
- Roblox Creator Hub: Discovery
<https://create.roblox.com/docs/discovery>
- Roblox Creator Hub: Creator Rewards
<https://create.roblox.com/docs/en-us/creator-rewards.md>
- Roblox Creator Hub: Developer Exchange
<https://create.roblox.com/docs/de-de/production/monetization/developer-exchange>
- Roblox Creator Hub: Tax information
<https://create.roblox.com/docs/production/monetization/tax-information>
- Roblox Form 10-Q (Q2 2026)
<https://www.sec.gov/Archives/edgar/data/0001315098/000162828026051082/rblx-20260630.htm>
- Roblox Newsroom: Cube Foundation Model / 4D generation (Feb 2026)
<https://about.roblox.com/newsroom/2026/02/accelerating-creation-powered-roblox-cube-foundation-model>
- Roblox: Altersprüfung für Chat (Jan 2026)
<https://www.nasdaq.com/press-release/roblox-requires-users-worldwide-age-check-access-chat-2026-01-07>
- Dexerto: Top-Creator-Verdienste
<https://www.dexerto.com/roblox/roblox-top-creators-average-65-7-million-a-year-but-most-make-far-less-3408675/>
- TweakTown: Median-Verdienst
<https://www.tweaktown.com/news/113585/roblox-top-10-creators-averaged-dollars65-7-million-each-while-the-median-made-dollars1500-and-roblox-everywhere-is-the-fix/index.html>
- IBTimes: DevEx-Auszahlungen 2026
<https://www.ibtimes.co.uk/roblox-top-creators-devex-payouts-2026-1819403>
- PocketGamer.biz: Steal a Brainrot 25 Mio. CCU
<https://www.pocketgamer.biz/roblox-steal-a-brainrot-becomes-first-game-to-surpass-25m-concurrent-players>
- GameAnalytics: 2025 Roblox Report
<https://www.gameanalytics.com/cn/reports/2025-roblox-report>
- RoWatcher: Launch-Playbook 2026
<https://rowatcher.com/news/how-to-launch-a-roblox-game-in-2026-the-pre-launch-playbook>
- RoWatcher: DevEx-Kurs (umstritten)
<https://rowatcher.com/news/the-real-roblox-devex-rate-in-2026-it-s-not-0-0035>
- RoLearn: DevEx-Anforderungen (umstritten)
<https://rolearn.dev/guidance/roblox-devex-requirements-2026>

Technische Hinweise zum Starter-Code

- Repo: `game/` (Rojo-Projekt, `src/`, `tests/`); Tests laufen mit einer Luau-Laufzeit (siehe `game/tests/README.md`).

- Dieses PDF wird aus `docs/handbuch/src/` erzeugt und kann mit `python3 docs/handbuch/src/build.py` neu gebaut werden.
- Alle Luau-Beispiele im Handbuch wurden mit einem Luau-Parser auf Syntax geprüft.